



郑州大学
Zhengzhou University

第三单元 序列密码

郑州大学网络空间安全学院

王志华

王志华



第3单元 序列密码

提纲

CONTENTS

3.1 序列密码概述

3.2 序列密码基础

3.3 典型序列密码

2022/4/9



3、线性反馈移位寄存器概述：定义

- 如果移位寄存器的**反馈函数** $f(a_1, a_2, \dots, a_n)$ 是 $a_1, a_2, \dots, a_n$ 的**线性函数**，则称之为**线性反馈移位寄存器LFSR**
(Linear Feedback Shift Register)

➤ 此时 f 可**写为**：

$$f(a_1, a_2, \dots, a_n) = c_n a_1 \oplus c_{n-1} a_2 \oplus \dots \oplus c_1 a_n$$

- ① 常数 $c_i = 0$ 或 1 ， $\oplus$ 是模2加法
- ② $c_i = 0$ 或 1 ，可用开关的断开和闭合来实现

2022/4/9



3、线性反馈移位寄存器概述：输出

$$f(a_1, a_2, \dots, a_n) = c_1 a_n \oplus c_2 a_{n-1} \oplus \dots \oplus c_n a_1$$

$$a_{n+1} = c_1 a_n \oplus c_2 a_{n-1} \oplus \dots \oplus c_n a_1$$

$$a_{n+2} = c_1 a_{n+1} \oplus c_2 a_n \oplus \dots \oplus c_n a_2$$

.....

输出序列 $\{a_t\}$ 满足以下公式，其中 t 为非负正整数

$$a_{n+t} = c_n a_t \oplus c_{n-1} a_{t+1} \oplus \dots \oplus c_1 a_{n+t-1}$$

- **线性反馈移位寄存器**因其实现简单、速度快、有较为成熟的理论等优点而成为构造密钥流生成器的最重要的部件之一。

王华



5、线性反馈移位寄存器性质

总是假定 $c_1, c_2, \dots, c_n$ 中至少有一个不为0，否则 $f(a_1, a_2, \dots, a_n) \equiv 0$ 。

总是假定 $c_n=1$ 。

- LFSR输出序列的性质：完全由其反馈函数决定。
- n 级LFSR状态数：最多有 2^n 个
- n 级LFSR的状态周期： $\leq 2^n - 1$
- 输出序列的周期=状态周期， $\leq 2^n - 1$
- 选择合适的反馈函数可使序列的周期达到最大值 $2^n - 1$ ，周期达到最大值的序列称为m序列。

王华

3.2 序列密码基础



3.2.2 m序列

定义2.1 设n级线性移位寄存器的输出序列满足递推关系

$$a_{n+k} = c_1 a_{n+k-1} \oplus c_2 a_{n+k-2} \oplus \dots \oplus c_n a_k \quad (*)$$

用延迟算子 D ($Da_k = a_{k-1}$) 作为未定元, 给出的反馈多项式为:

$$p(D) = 1 + c_1 D + \dots + c_{n-1} D^{n-1} + c_n D^n$$

这种递推关系可用一个一元高次多项式

$$p(x) = 1 + c_1 x + \dots + c_{n-1} x^{n-1} + c_n x^n$$

表示, 称这个多项式为LFSR的特征多项式。

延迟算子 (Backshift Operator) 也称为滞后算子 (Lag Operator), 一般记为 B , 作用于时间序列的时间指标上, 将时间向后倒退一个时间单位, 形成一个新的序列。

2022/4/9



1、LFSR的一元特征多项式：推理

$$a_{n+k} = c_1 a_{n+k-1} \oplus c_2 a_{n+k-2} \oplus \cdots \oplus c_n a_k$$

$$\Leftrightarrow a_{n+k} \oplus c_1 a_{n+k-1} \oplus c_2 a_{n+k-2} \oplus \cdots \oplus c_n a_k = 0$$

$D(a_k) = a_{k-1}$ ，用 $p(D) = 1 + c_1 D + \cdots + c_n D^n$ 作用于 a_{n+k} 后恰好就是上式的左边，

即

$$\begin{aligned} p(D)(a_{n+k}) &= (1 + c_1 D + \cdots + c_n D^n)(a_{n+k}) \\ &= a_{n+k} + c_1 D(a_{n+k}) + \cdots + c_n D^n(a_{n+k}) \\ &= a_{n+k} + c_1 a_{n+k-1} + c_2 a_{n+k-2} + \cdots + c_n a_k \end{aligned}$$

2022/4/9



1、LFSR的一元特征多项式：非零序列

设 n 级线性移位寄存器对应于递推关系 $(*)$ ，由于 $a_i \in GF(2) (i=1, 2, \dots, n)$ ，所以共有 2^n 组初始状态，即有 2^n 个递推序列，其中非恒零的有 $2^n - 1$ 个，记 $2^n - 1$ 个非零序列的全体为 $G(p(x))$ 。

2022/4/9



1、LFSR的一元特征多项式：生成函数

定义2.2 给定序列 $\{a_i\}$ ，幂级数

$$A(x) = \sum_{i=1}^{\infty} a_i x^{i-1}$$

称为该序列的生成函数。

2022/4/9



1、LFSR的一元特征多项式：生成函数

定理2.1 设 $p(x)=1+c_1x+\dots+c_{n-1}x^{n-1}+c_nx^n$ 是 $GF(2)$ 上的多项式， $G(p(x))$ 中任一序列 $\{a_i\}$ 的生成函数 $A(x)$ 满足：

$$A(x) = \frac{\phi(x)}{p(x)}$$

其中

$$\phi(x) = \sum_{i=1}^n (c_{n-i} x^{n-i} \sum_{j=1}^i a_j x^{j-1})$$

2022/4/9



2、LFSR的一元特征多项式的相关定理

定理2.2 $p(x)|q(x)$ 的充要条件是 $G(p(x)) \subset G(q(x))$ 。

——该定理说明：可用 n 级LFSR产生的序列，也可用级数更多的LFSR来产生。

定义2.3 设 $p(x)$ 是 $GF(2)$ 上的多项式，使 $p(x)|(x^p-1)$ 成立的最小正整数 p 称为 $p(x)$ 的周期或阶。

定理2.3 若序列 $\{a_i\}$ 的特征多项式 $p(x)$ 定义在 $GF(2)$ 上， p 是 $p(x)$ 的周期，则 $\{a_i\}$ 的周期 $r|p$ 。

——该定理说明： n 级LFSR输出序列的周期 r ，不依赖于初始条件，而依赖于特征多项式 $p(x)$ 。

2022/4/9



2、LFSR的一元特征多项式的相关定理

定义2-3

仅能被非0常数或自身的常数倍除尽，但不能被其它多项式除尽的多项式称为**即约多项式**或**不可约多项式**。

定理2-4

设 $p(x)$ 是 n **次不可约多项式**，**周期为** m ，序列 $\{a_i\} \in G(p(x))$ ，**则** $\{a_i\}$ **的周期为** m 。

2022/4/9



3、LFSR的输出序列为m序列：必要条件

定理2.5 n 级LFSR产生的序列有最大周期 2^n-1 的必要条件是其特征多项式为不可约的。

证明：设 n 级LFSR产生的序列周期达到最大 2^n-1 。

反证法：设特征多项式为 $p(x)$ ，若 $p(x)$ 可约，可设为 $p(x)=g(x)h(x)$ ，其中 $g(x)$ 不可约，且次数 $k < n$ 。由于 $G(g(x)) \subset G(p(x))$ ，而 $G(g(x))$ 中序列的周期一方面不超过 2^k-1 ，另一方面又等于 2^n-1 ，这是矛盾的，所以 $p(x)$ 不可约。

该定理的逆不成立，即LFSR的特征多项式为不可约多项式时，其输出序列不一定是m序列。

王华



3、LFSR的输出序列为m序列：反例示例

例2-4

$f(x) = x^4 + x^3 + x^2 + x + 1$ 为 $GF(2)$ 上的不可约多项式，这是因为一次多项式 $x, x+1$ 都不能整除，因此任一三次多项式也不能整除 $f(x)$ 。而二次多项式有 $x^2 = x \cdot x, x^2 + 1 = (x+1)(x+1)$ （在 $GF(2)$ 上 $x+x=0$ ）， $x^2 + x = x(x+1), x^2 + x + 1$ 。由 $x, x+1$ 都不能整除 $f(x)$ 知 $x^2, x^2 + 1, x^2 + x$ 都不能整除，二次不可约多项式 $x^2 + x + 1$ 不能整除 $f(x)$ 可直接验证。

以 $f(x)$ 为特征多项式的LFSR的输出序列可由

$$a_k = a_{k-1} \oplus a_{k-2} \oplus a_{k-3} \oplus a_{k-4} \quad (k \geq 4)$$

和给定的初始状态求出，设初始状态为0001，则输出序列为00011000110001...，周期为5，不是 m 序列。

2022/4/9



3、LFSR的输出序列为m序列：充要条件

定义2.4 若 n 次不可约多项式 $p(x)$ 的阶为 2^n-1 ，则称 $p(x)$ 是 n 次本原多项式。

定理2.6 设 $\{a_i\} \in G(p(x))$ ， $\{a_i\}$ 为 m 序列的充要条件是 $p(x)$ 为本原多项式。

证明：若 $p(x)$ 是本原多项式，则其阶为 2^n-1 ，得 $\{a_i\}$ 的周期等于 2^n-1 ，即 $\{a_i\}$ 为 m 序列。

反之，若 $\{a_i\}$ 为 m 序列，即其周期等于 2^n-1 ，由定理2.5知 $p(x)$ 是不可约的。由定理2.3知 $\{a_i\}$ 的周期 2^n-1 整除 $p(x)$ 的阶，而 $p(x)$ 的阶不超过 2^n-1 ，所以 $p(x)$ 的阶为 2^n-1 ，即 $p(x)$ 是本原多项式。

对于任意的正整数 n ，至少存在一个 n 次本原多项式。所以对于任意的 n 级LFSR，至少存在一种连接方式使其输出序列为 m 序列

思考：递推关系式和特征多项式之间如何相互转换？

王华



3、LFSR的输出序列为m序列：正例示例

例2-5

设 $p(x) = x^4 + x + 1$ ，由于 $p(x) \nmid (x^{15} - 1)$ ，但不存在小于15的常数 l ，使得 $p(x) \mid (x^l - 1)$ ，所以 $p(x)$ 的阶为15。类似于例2-4， $p(x)$ 的不可约性可由 $x, x+1, x^2+x+1$ 都不能整除 $p(x)$ 得到，所以 $p(x)$ 是本原多项式。

若LFSR以 $p(x)$ 为特征多项式，则输出序列的递推关系为

$$a_k = a_{k-1} \oplus a_{k-4} \quad (k \geq 4)$$

若初始状态为1001，则输出为

10010001111010110010001111010

周期为 $2^4 - 1 = 15$ ，即输出序列为 m 序列。

2022/4/9



4、m序列的伪随机性：概述

- 序列密码的安全性取决于**密钥流的安全性**，要求密钥流序列有**好的随机性**，以使密码分析者对它**无法预测**。也就是说，即使截获其中一段，也无法推测后面是什么。
 - 如果密钥流是周期的，要完全做到随机性是困难的。
 - 严格地说，**周期性的序列不可能做到随机**，只能要求截获比周期短的一段时不会泄露更多信息，这样的序列称为**伪随机序列**。

王卫华

4、m序列的伪随机性：游程、自相关



游程： 设 $\{a_i\}=(a_1a_2a_3\cdots)$ 为二元序列，例如**00110111**，其前两个数字是00，称为0的2游程；接着是11，是1的2游程；再下来是0的1游程和1的3游程。

自相关函数： GF(2) 上周期为T的序列 $\{a_i\}$ 的自相关函数定义为

$$R(t) = \sum_{k=1}^T (-1)^{a_k} (-1)^{a_{k+t}}, 0 \leq t \leq T-1$$

当 $t=0$ 时， $R(t)=T$ ；当 $t \neq 0$ 时，称 $R(t)$ 为**异相自相关函数**。

王华



4、m序列的伪随机性：伪随机假设

3个随机性公设：

- ① 在序列的一个周期内，0与1的个数相差至多为1。
 - 说明 $\{a_i\}$ 中0与1出现的概率基本上相同
- ② 在序列的一个周期内，长为 i 的游程占游程总数的 $1/2^i$ ($i=1,2,\dots$)，且在等长的游程中0的游程个数和1的游程个数相等。
 - 说明0与1在序列中每一位置上出现的概率相同
- ③ 异相自相关函数是一个常数。
 - 意味着通过对序列与其平移后的序列做比较，不能给出其他任何信息

王华

4、m序列的伪随机性：伪随机序列定义



设 $\underline{a} = (a_1, a_2, \dots, a_{p(\underline{a})}, \dots)$ 是 $GF(2)$ 上一个周期等于 $p(\underline{a})$ 的周期序列。

如果对于一切 $t \not\equiv 0 \pmod{p(\underline{a})}$ ，有

$$R(t) = -1$$

则称序列 $\underline{a} = (a_1, a_2, \dots, a_{p(\underline{a})}, \dots)$ 为伪随机序列。

2022/4/9



4、m序列的伪随机性：伪随机序列条件

从密码系统的角度看，一个伪随机序列还应满足下面的条件：

- $\{a_i\}$ 的周期相当大。
- $\{a_i\}$ 的确定是计算上容易的。
- $\{a_i\}$ 由密文及相应的明文的部分信息,不能确定整个 $\{a_i\}$ 。

下一定理说明， m 序列满足Golomb的三个随机性公设。

王华



4、m序列的伪随机性：定理

m序列满足**Golomb**的**3**个随机性公设。

定理2.7 **GF(2)**上的**n**长**m**序列 $\{a_i\}$ 具有如下性质：

- ① 在一个周期内，**0**、**1**出现的次数分别为 $2^{n-1}-1$ 和 2^{n-1} 。
- ② 在一个周期内，总游程数为 2^{n-1} ；对 $1 \leq i \leq n-2$ ，长为**i**的游程有 2^{n-i-1} 个，且**0**、**1**游程各半；长为**n-1**的**0**游程一个，长为**n**的**1**游程一个。
- ③ $\{a_i\}$ 的自相关函数为

$$R(t) = \begin{cases} 2^n - 1, & t = 0 \\ -1, & 0 < t \leq 2^n - 2 \end{cases}$$

2022/4/9



5、m序列的递推关系式

- 寻找m序列的递推关系式。
 - 已知一段序列，如果知道其反馈多项式，就可以将其后的序列依次求出
 - 已知序列能否获得相应的反馈多项式(或线性递推式)呢？
 - 解方程方法——已知序列 $\{a_i\}$ 是由n级线性移存器产生的，并且知道 $\{a_i\}$ 的连续 $2n$ 位，可用解线性方程组的方法得到反馈多项式
 - 线性反馈移位寄存器综合解——Berlekamp-Massey算法

2022/4/9



5、m序列的递推关系式：解方程法示例1

例1 设序列 $a = (01111000\dots)$ 是由4级线性移存器所产生序列的连续8个信号，求该移存器的线性递推式。

解：设该4级移存器的线性递推式为：

$$a_n = c_1 a_{n-1} \oplus c_2 a_{n-2} \oplus c_3 a_{n-3} \oplus c_4 a_{n-4} \quad (n \geq 4)$$

由于知道周期序列的连续8个信号，不妨设为开头的8个信号，即

$$a_0 a_1 a_2 a_3 a_4 a_5 a_6 a_7 = 01111000$$

当 $n=4$ 时，由递归式可得： $a_4 = c_1 a_3 \oplus c_2 a_2 \oplus c_3 a_1 \oplus c_4 a_0$

即：

$$c_1 \oplus c_2 \oplus c_3 = 1 \quad (1)$$

同理可得：

$$c_1 \oplus c_2 \oplus c_3 \oplus c_4 = 0 \quad (2)$$

$$c_2 \oplus c_3 \oplus c_4 = 0 \quad (3)$$

$$c_3 \oplus c_4 = 0 \quad (4)$$

解方程组得 $c_1 = 0, c_2 = 0, c_3 = 1, c_4 = 1$

故所求移存器递推式为：

$$a_n = a_{n-3} \oplus a_{n-4} \quad (n \geq 4)$$



5、m序列的递推关系式：解方程法示例2

例 设敌手得到密文串：**101101011110010**

和相应的明文串：**011001111111001**

因此，可得相应的密钥流：**110100100001011**

进一步假定敌手还知道密钥流是使用**5**级线性反馈移位寄存器产生的，那么敌手可分别用密钥流中的前**10**个比特建立如下方程

$$(a_6 \ a_7 \ a_8 \ a_9 \ a_{10}) = (c_5 \ c_4 \ c_3 \ c_2 \ c_1) \begin{pmatrix} a_1 & a_2 & a_3 & a_4 & a_5 \\ a_2 & a_3 & a_4 & a_5 & a_6 \\ a_3 & a_4 & a_5 & a_6 & a_7 \\ a_4 & a_5 & a_6 & a_7 & a_8 \\ a_5 & a_6 & a_7 & a_8 & a_9 \end{pmatrix}$$

2024



5、m序列的递推关系式：解方程法示例2

即

$$(0 \ 1 \ 0 \ 0 \ 0) = (c_5 \ c_4 \ c_3 \ c_2 \ c_1) \begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

而

$$\begin{pmatrix} 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}^{-1} = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

2022/4/9



5、m序列的递推关系式：解方程法示例2

从而得到

$$(c_5 \ c_4 \ c_3 \ c_2 \ c_1) = (0 \ 1 \ 0 \ 0 \ 0) \begin{pmatrix} 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

所以

$$(c_5 \ c_4 \ c_3 \ c_2 \ c_1) = (1 \ 0 \ 0 \ 1 \ 0)$$

密钥流的递推关系为

$$a_{i+5} = c_5 a_i \oplus c_2 a_{i+3} = a_i \oplus a_{i+3}$$

2022/4/9



6、m序列的递推关系式：LFSR的两个问题

根据密码学的需要，对线性反馈移位寄存器(LFSR)主要考虑下面两个问题：

(1) 如何利用级数尽可能短的LFSR产生周期大、随机性能良好的序列。

这是从密钥生成角度考虑，用最小的代价产生尽可能好的、参与密码变换的序列。

(2) 当已知一个长为 N 序列 $\underline{a}$ 时，如何构造一个级数尽可能小的LFSR来产生它。

这是从密码分析角度来考虑，要想用线性方法重构密钥序列所必须付出的最小代价。

王卫华



6、m序列的递推关系式：线性综合解的定义

设 $\underline{a} = (a_0, a_1, \dots, a_{N-1})$ 是 F_2 上的长度为 N 的序列，而

$f(x) = c_0 + c_1x + c_2x^2 + \dots + c_lx^l$ 是 F_2 上的多项式， $c_0=1$ 。

如果序列中的元素满足递推关系：

$$a_k = c_1a_{k-1} \oplus c_2a_{k-2} \oplus \dots \oplus c_la_{k-l}, k = l, l+1, \dots, N-1 \quad (2)$$

则称 $\langle f(x), l \rangle$ 产生二元序列 $\underline{a}$ 。其中 $\langle f(x), l \rangle$ 表示以 $f(x)$ 为特征多项式的 l 级线性移位寄存器。

如果 $f(x)$ 是一个能产生 $\underline{a}$ 并且级数最小的线性移位寄存器的特征多项式， l 是该寄存器的级数，则称 $\langle f(x), l \rangle$ 为序列 $\underline{a}$ 的线性综合解。

2022/4/9



6、m序列的递推关系式：B-M法问题定义

线性移位寄存器的综合问题可表述为：给定一个 N 长二元序列 $\underline{a}$ ，如何求出产生这一序列的最小级数的线性移位寄存器，即最短的线性移存器。

1、特征多项式 $f(x)$ 的次数 $\leq l$ 。因为产生 $\underline{a}$ 且级数最小的线性移位寄存器可能是退化的，在这种情况下 $f(x)$ 的次数 $< l$ ；并且此时 $f(x)$ 中的 $c_l=0$ ，因此在特征多项式 $f(x)$ 中仅要求 $c_0=1$ ，但不要要求 $c_l=1$ 。

2、规定：0级线性移位寄存器是以 $f(x)=1$ 为特征多项式的线性移位寄存器，且 n 长($n=1, 2, \dots, N$)全零序列，仅由0级线性移位寄存器产生。事实上，以 $f(x)=1$ 为反馈特征多项式的递归关系式是： $a_k=0, k=0, 1, \dots, n-1$ 。因此，这一规定是合理的。

3、给定一个 N 长二元序列 $\underline{a}$ ，求能产生 $\underline{a}$ 并且级数最小的线性移位寄存器，就是求 $\underline{a}$ 的线性综合解。利用B-M算法可以有效的求出。

2022/4/9



6、m序列的递推关系式：B-M算法

用归纳法求出一系列线性移位寄存器：

$$\langle f_n(x), l_n \rangle \quad \partial^0 f_n(x) \leq l_n, \quad n=1, 2, \dots, N$$

每一个 $\langle f_n(x), l_n \rangle$ 都是产生序列 $\underline{a}$ 的前 n 项的最短线性移位寄存器，在 $\langle f_n(x), l_n \rangle$ 的基础上构造相应的 $\langle f_{n+1}(x), l_{n+1} \rangle$ ，使得是 $\langle f_{n+1}(x), l_{n+1} \rangle$ 产生给定序列前 $n+1$ 项的最短移存器，则最后得到的 $\langle f_N(x), l_N \rangle$ 就是产生给定 N 长二元序列 $\underline{a}$ 的最短的线性移位寄存器。

2022/4/9



6、m序列的递推关系式：B-M算法

任意给定一个N长序列 $\underline{a} = (a_0, a_1, \dots, a_{N-1})$ ，按n归纳定义

$$\langle f_n(x), l_n \rangle \quad n = 0, 1, 2, \dots, N-1$$

1、取初始值： $f_0(x) = 1, l_0 = 0$

2、设 $\langle f_0(x), l_0 \rangle, \langle f_1(x), l_1 \rangle, \dots, \langle f_n(x), l_n \rangle$ ($0 \leq n < N$) 均已求得，

且 $l_0 \leq l_1 \leq \dots \leq l_n$

记： $f_n(x) = c_0^{(n)} + c_1^{(n)}x + \dots + c_{l_n}^{(n)}x^{l_n}, c_0^{(n)} = 1$ ，再计算：

$$d_n = c_0^{(n)}a_n + c_1^{(n)}a_{n-1} + \dots + c_{l_n}^{(n)}a_{n-l_n}$$

称 d_n 为第n步差值。然后分两种情形讨论：

2022/4/9



6、m序列的递推关系式：B-M算法

(i) 若 $d_n=0$ ，则令：

$$f_{n+1}(x) = f_n(x), \quad l_{n+1} = l_n。$$

(ii) 若 $d_n=1$ ，则需区分以下两种情形：

① 当： $l_0 = l_1 = \dots = l_n = 0$ 时，

$$\text{取：} f_{n+1}(x) = 1 + x^{n+1}, l_{n+1} = n + 1。$$

② 当有 m ($0 \leq m < n$)，使： $l_m < l_{m+1} = l_{m+2} = \dots = l_n$ 。

$$\text{便置：} f_{n+1}(x) = f_n(x) + x^{n-m} f_m(x), l_{n+1} = \max\{l_n, n + 1 - l_n\}$$

最后得到的 $\langle f_N(x), l_N \rangle$ 便是产生序列 $\underline{a}$ 的最短线性移位寄存器。

2022/4/9



6、m序列的递推关系式：B-M算法示例

输入：S⁸=10101111

n	d _n	f _n	L _n	m	f _m
0	1	1	0		
1	1	1 + x	1	0	1
2	1	1	1	0	1
3	0	1 + x ²	2		
4	0	1 + x ²	2		
5	1	1 + x ²	2	2	1
6	0	1 + x ² + x ³	4		
7	1	1 + x ² + x ³	4	5	1 + x ²
8		1 + x ³ + x ⁴	4		

输出：<1+x³+x⁴, 4>

$$f_n(x) = c_0^{(n)} + c_1^{(n)}x + \cdots + c_{l_n}^{(n)}x^{l_n}, c_0^{(n)} = 1,$$

$$d_n = c_0^{(n)}a_n + c_1^{(n)}a_{n-1} + \cdots + c_{l_n}^{(n)}a_{n-l_n}$$

(i) 若 $d_n = 0$, 则令:

$$f_{n+1}(x) = f_n(x), \quad l_{n+1} = l_n.$$

(ii) 若 $d_n = 1$, 则需区分以下两种情形:

① 当: $l_0 = l_1 = \cdots = l_n = 0$ 时,

$$\text{取: } f_{n+1}(x) = 1 + x^{n+1}, l_{n+1} = n+1.$$

② 当有 m ($0 \leq m < n$), 使:

$$\text{③ } l_m < l_{m+1} = l_{m+2} = \cdots = l_n.$$

$$\text{便置: } f_{n+1}(x) = f_n(x) + x^{n-m}f_m(x), \\ l_{n+1} = \max\{l_n, n+1-l_n\}$$

2022/4/9

3.2 序列密码基础



3.2.3 非线性组合

将LFSR的输出序列直接作为密钥序列，即将LFSR作为密钥序列产生器，可行吗？

50年代开始用作密钥序列，并用于军用；
60年代发现其是不安全的

- 结论：
 - 单一使用LFSR并不安全，容易被密码分析。
 - 需要进行非线性组合。

王华



1、密钥流生成器：需求

- 为了使密钥流生成器输出的**二元序列尽可能复杂**，应保证：
 - 较好的统计特性，不可预测性尽可能高
 - 其周期尽可能大
 - 具有大线性复杂度
- 使用**多个LFSR**来构造**二元序列**
 - 称每个LFSR的输出序列为**驱动序列**
- **非线性组合密钥流生成器**输出序列的周期不大于**各驱动序列周期的乘积**
 - 提高输出序列的线性复杂度应从**极大化其周期开始（如何最大化？）**。

王卫华



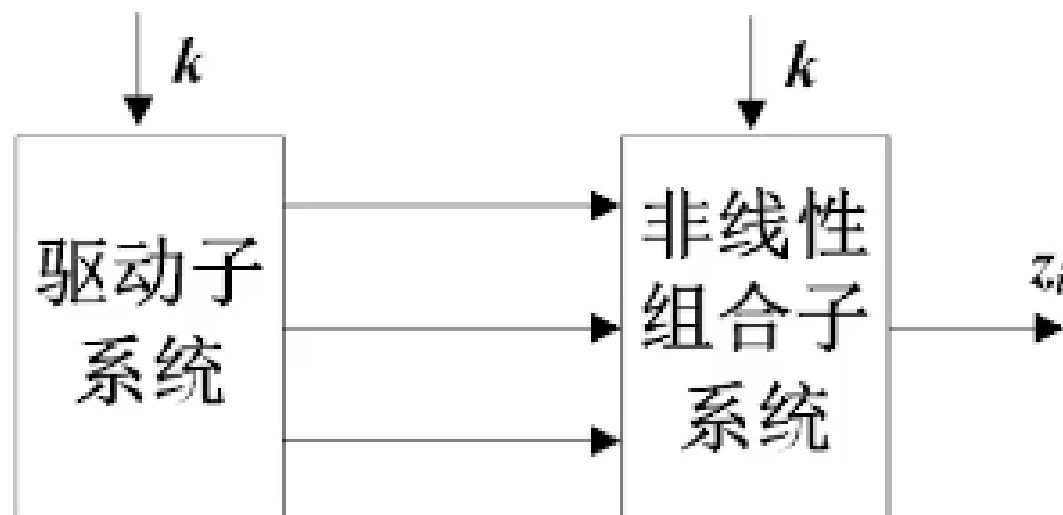
1、密钥流生成器：组成

- 密钥流生成器可分解为**驱动子系统**和**非线性组合子系统**
 - **驱动子系统**常用一个或多个线性反馈移位寄存器来实现
 - **非线性组合子系统**用非线性组合函数来实现

王华



1、密钥流生成器：组成

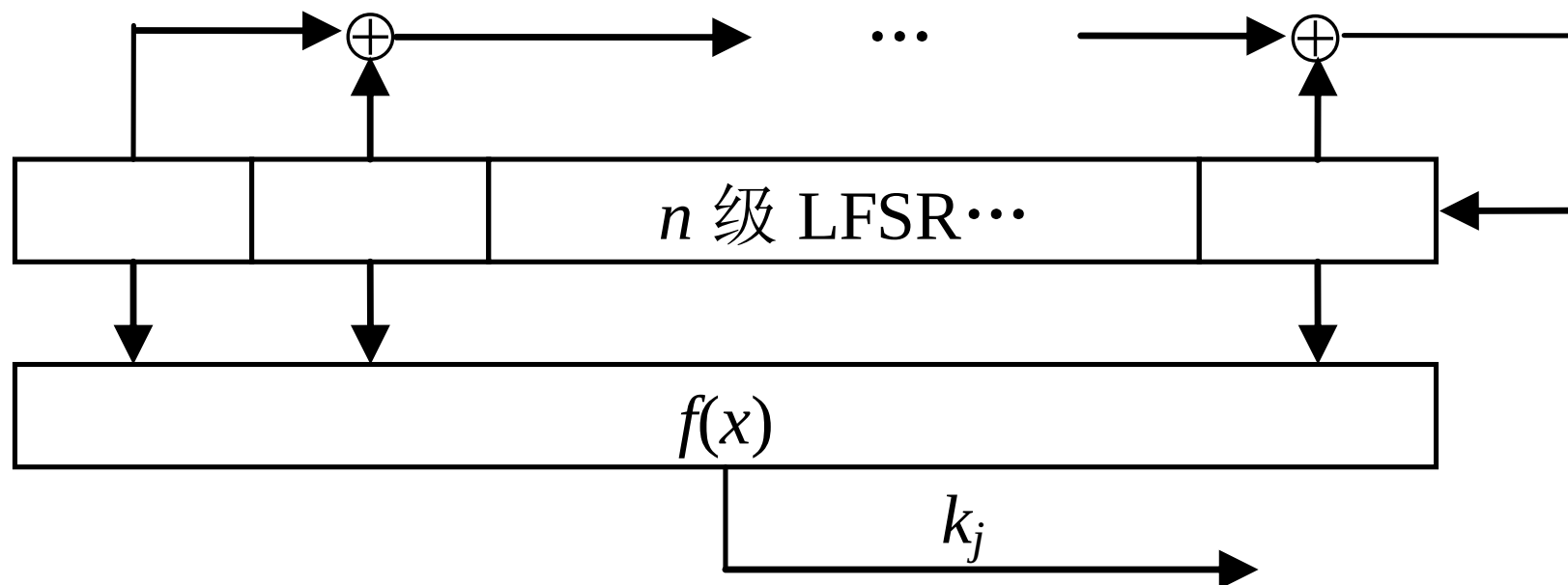


密钥流生成器的分解

王华



1、密钥流生成器：非线性前馈序列生成器



2、Geffe序列生成器

- Geffe序列生成器也可以表示为图2-13的形式，其中LFSR1 和LFSR3作为多路复合器的输入，LFSR2控制多路复合器的输出。

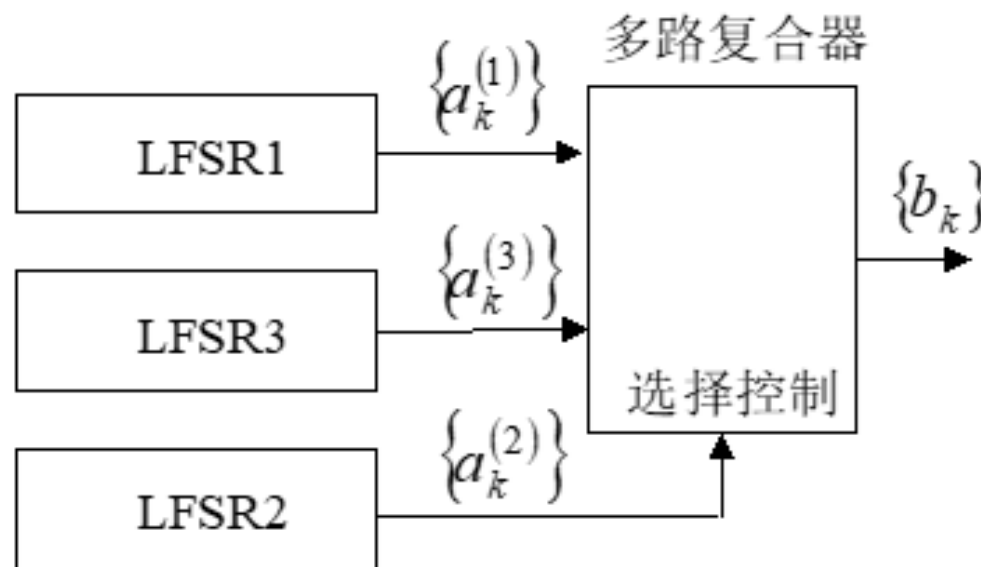
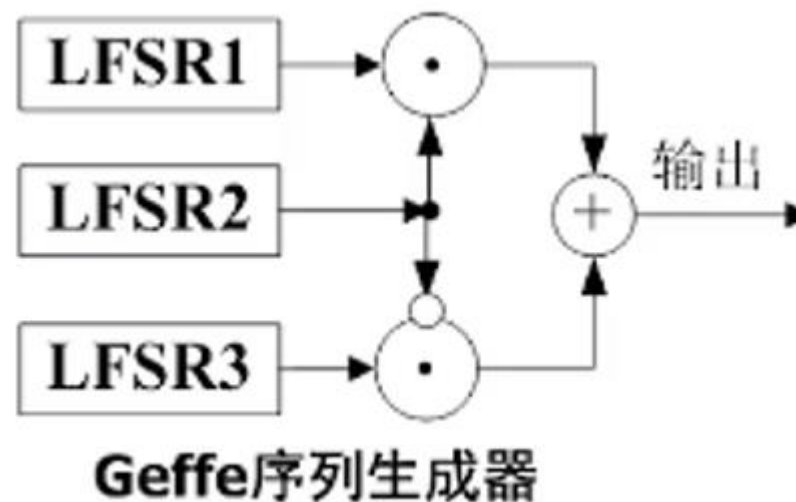


图2-13多路复合器表示的Geffe序列生成器



2、Geffe序列生成器

- **Geffe**序列生成器由3个LFSR组成，其中LFSR2作为控制生成器使用，如图所示



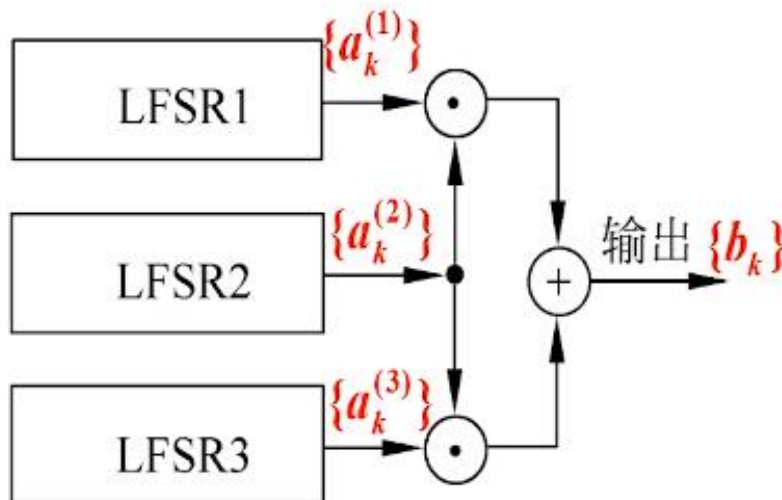
- 当LFSR2输出1时，LFSR2与LFSR1相连接
- 当LFSR2输出0时，LFSR2与LFSR3相连接

如何设计？

2022/4/9



2、Geffe序列生成器



若设LFSR i 的输出序列为 $\{a_k^{(i)}\}$ ($i=1,2,3$),
则输出序列 $\{b_k\}$ 可以表示为

$$b_k = a_k^{(1)} a_k^{(2)} + a_k^{(3)} \overline{a_k^{(2)}} = a_k^{(1)} a_k^{(2)} + a_k^{(3)} a_k^{(2)} + a_k^{(3)}$$

设LFSR i 的特征多项式分别为 n_i 次本原多项式, 且 n_i 两两互素

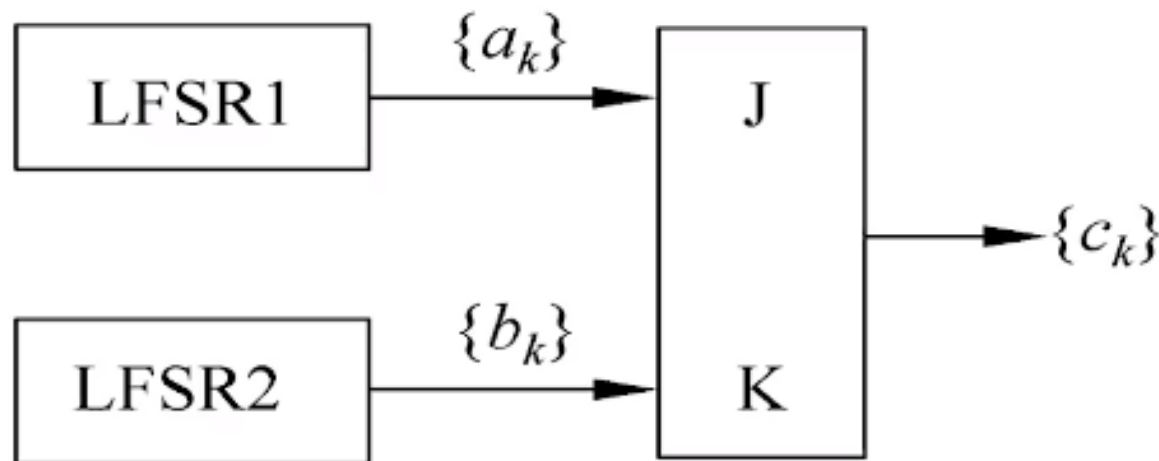
则Geffe序列的周期 =
$$\prod_{i=1}^3 (2^{n_i} - 1)$$

线性复杂度 =
$$(n_1 + n_3) n_2 + n_3$$

2022/4/9



3、J-K触发器：非线性序列生成器



$\{a_k\}$: m级m序列

$\{b_k\}$: n级m序列

2022/4/9

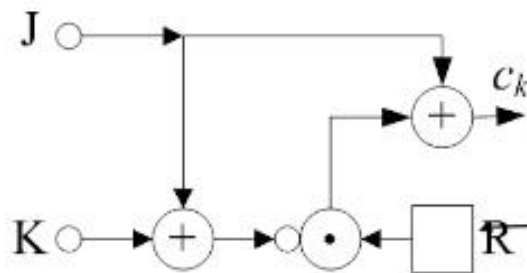


3、J-K触发器

J-K触发器如图所示，它的两个输入端分别用J和K表示，其输出 c_k 不仅依赖于输入，还依赖于前一个输出位 c_{k-1} ，即

$$c_k = \overline{(x_1 + x_2)} c_{k-1} + x_1$$

其中 x_1 和 x_2 分别是J和K端的输入。由此可得J-K触发器的真值表，如下表所示



J-K触发器

J	K	c_k
0	0	c_{k-1}
0	1	0
1	0	1
1	1	$\overline{c_{k-1}}$

J-K触发器真值表

2022/4/9



3、J-K触发器：非线性序列生成器

利用J-K触发器的非线性序列生成器见图2-15，令驱动序列 $\{a_k\}$ 和 $\{b_k\}$ 分别为 m 级和 n 级 m 序列，则有

$$c_k = \overline{(a_k + b_k)}c_{k-1} + a_k = (a_k + b_k + 1)c_{k-1} + a_k$$

如果令 $c_{-1} = 0$ ，则输出序列的最初三项为：

$$c_0 = a_0$$

$$c_1 = (a_1 + b_1 + 1)a_0 + a_1$$

$$c_2 = (a_2 + b_2 + 1)((a_1 + b_1 + 1)a_0 + a_1) + a_2$$

当 m 与 n 互素且 $a_0 + b_0 = 1$ 时，序列 $\{c_k\}$ 的周期为 $(2^m - 1)(2^n - 1)$

2022/4/9



3、J-K触发器：非线性序列生成器示例

例2.7 令 $m=2, n=3$ ，两个驱动 m 序列分别为

$$\{a_k\}=0,1,1,\dots$$

和

$$\{b_k\}=1,0,0,1,0,1,1,\dots$$

于是，输出序列 $\{c_k\}$ 是 $0,1,1,0,1,0,0,1,1,1,0,1,0,1,0,0,1,0,0,1,0,\dots$ ，
其周期为 $(2^2-1)(2^3-1)=21$ 。

2022/4/9



3、J-K触发器：弱点

由 $c_k = (a_k + b_k + 1)c_{k-1} + a_k$ 可得

$$c_k = \begin{cases} a_k, & c_{k-1} = 0 \\ \overline{b_k}, & c_{k-1} = 1 \end{cases}$$

- 如果知道 $\{c_k\}$ 中相邻位的值 c_{k-1} 和 c_k ，就可以推断出 a_k 和 b_k 中的一个。而一旦知道足够多的这类信息，就可通过密码分析的方法得到序列 $\{a_k\}$ 和 $\{b_k\}$ 。
- 为了克服上述缺点，Pless提出了由多个J-K触发器序列驱动的多路复合序列方案，称为Pless生成器。

2022/4/9



4、Pless生成器:

Pless生成器由8个LFSR、4个J-K触发器和1个循环计数器构成，由循环计数器进行选通控制，如图2-16所示。假定在时刻 t 输出第 $t(\bmod 4)$ 个单元，则输出序列为

$$a_0 b_1 c_2 d_3 a_4 b_5 c_6 \dots$$

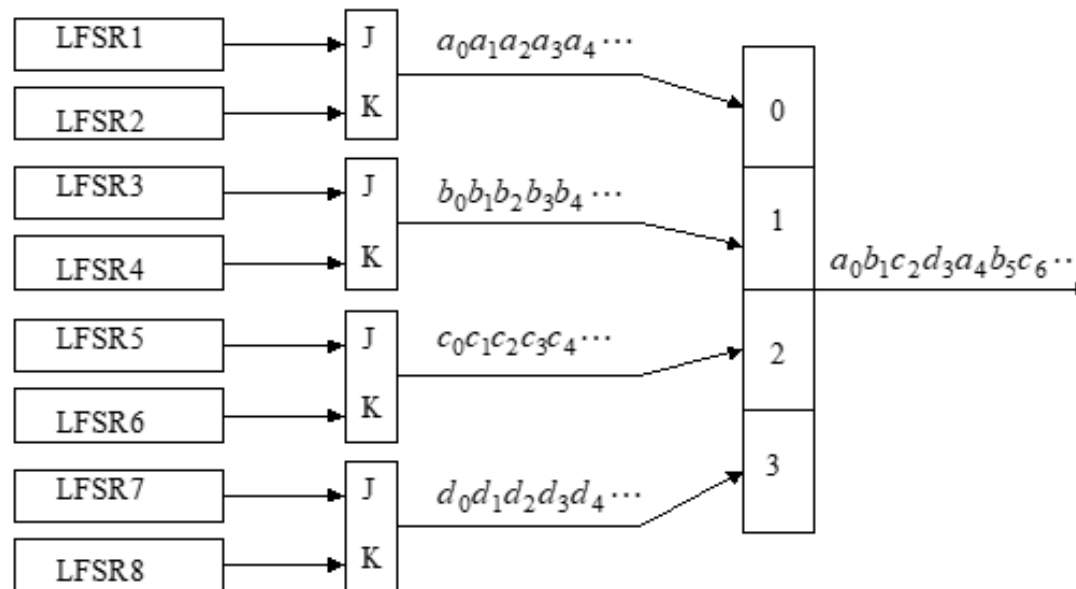


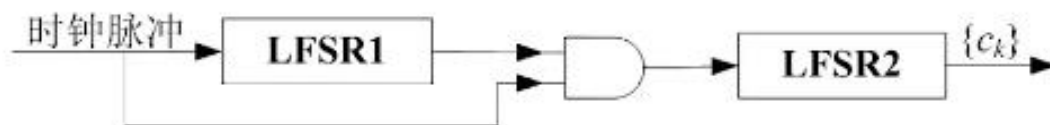
图2-16 Pless生成器

2022/4/9



5、钟控序列生成器：构成

- 钟控序列最基本的模型是用一个**LFSR**控制另外一个**LFSR**的移位时钟脉冲，如图所示，一个最简单钟控序列生成器



- 假设**LFSR1**和**LFSR2**分别输出序列 $\{a_k\}$ 和 $\{b_k\}$ ，其周期分别为 p_1 和 p_2 。
- 当**LFSR1**输出**1**时，移位时钟脉冲通过与门使**LFSR2**进行一次移位，从而生成下一位。
- 当**LFSR1**输出**0**时，移位时钟脉冲无法通过与门影响**LFSR2**。因此**LFSR2**重复输出前一位。

2022/4/9



5、钟控序列生成器：周期

- 假设LFSR1和LFSR2分别输出序列 $\{a_k\}$ 和 $\{b_k\}$ ，其周期分别为 p_1 和 p_2 。假设钟控序列 $\{c_k\}$ 的周期为 p ，可得如下关系：

- $$p = \frac{p_1 p_2}{\gcd(w_1, p_2)}, \quad \text{其中 } w_1 = \sum_{i=0}^{p_1-1} a_i$$

- c_k 的一个周期至少是LFSR1和LFSR2同时回到初始状态的时刻
- 显然当运行 $p_1 \times p_2$ 个节拍后两个LFSR必然回到初态，因此周期至多是 $p_1 \times p_2$
- LFSR1运行一个周期，LFSR2运行 $w_1 = dt$ 拍， $d = \gcd(w_1, p_2)$
- 则LFSR1运行 (p_2/d) 个周期后，LFSR2刚好运行 $dt \times p_2/d = tp_2$ 拍，即 t 个周期，于是两个LFSR都回到初态，这时运行了 $(p_2/d) \times p_1$ 个节拍

2022/4/9



5、钟控序列生成器：周期

- 若 $\{a_k\}$ 和 $\{b_k\}$ 的极小特征多项式分别为 $\text{GF}(2)$ 上的 m 和 n 次本原多项式 $f_1(x)$ 和 $f_2(x)$ ，且 $m|n$ 。
- 则 $p_1=2^m-1, p_2=2^n-1$ 。
- 而 w_1 为 $\{a_k\}$ 一个周期内1的个数，因此 $w_1=2^{m-1}$
- 故 $\text{gcd}(w_1, p_2)=1$ ，所以 $p=p_1p_2=(2^m-1)(2^n-1)$ 。

2022/4/9



5、钟控序列生成器：复杂度

- 可推导出 $\{c_k\}$ 的线性复杂度为 $n(2^m-1)$ ，极小特征多项式为 $f_2(x^{2^m-1})$
 - 其对应的LFSR2的抽头每隔周期 $p_1=2^m-1$ 一个，这样，参与运算的每个抽头对应的状态的节奏相同，从而相当于对LFSR2序列进行每 2^m-1 拍的抽样序列(不计由于LFSR1的0游程而产生的重复)，这个序列只是LFSR2的平移和按照LFSR1中的0游程进行迟延，而抽头应该与LFSR2的节奏一致，所以其极小多项式和线性复杂度如上

2022/4/9



5、钟控序列生成器：示例

- 例：设LFSR1为3级m序列生成器，其特征多项式为 $f_1(x)=1+x+x^3$ 。设初态为 $a_0=a_1=a_2=1$ ，于是输出序列为 $\{a_k\}=1,1,1,0,1,0,0,\dots$
- 又设LFSR2为3级m序列生成器，且记其状态向量为 σ_k ，则在上图的构造下 σ_k 的变化情况如下：
 - $\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_3, \sigma_4, \sigma_4, \sigma_4,$
 - $\sigma_5, \sigma_6, \sigma_0, \sigma_0, \sigma_1, \sigma_1, \sigma_1,$
 - $\sigma_2, \sigma_3, \sigma_4, \sigma_4, \sigma_5, \sigma_5, \sigma_5,$
 - $\sigma_6, \sigma_0, \sigma_1, \sigma_1, \sigma_2, \sigma_2, \sigma_2,$
 - $\sigma_0, \sigma_1, \sigma_2, \sigma_2, \sigma_3, \sigma_3, \sigma_3,$
 - $\sigma_4, \sigma_5, \sigma_6, \sigma_6, \sigma_0, \sigma_0, \dots$
- $\{c_k\}$ 的周期为 $(2^3-1)^2=49$ ，在它的一个周期内，每个 σ_k 恰好出现7次

- 当LFSR1输出1时，移位时钟脉冲通过与门使LFSR2进行一次移位，从而生成下一位。
- 当LFSR1输出0时，移位时钟脉冲无法通过与门影响LFSR2。因此LFSR2重复输出前一位。

2022



5、钟控序列生成器：示例

- 设 $f_2(x)=1+x^2+x^3$ 为 LFSR2 的特征多项式，且初态为 $b_0=b_1=b_2=1$ ，则 $\{b_k\}=1,1,1,0,0,1,0,1,1,1\dots$

- 由 σ_k 的变化情况得 $\{c_k\}=1,1,1,0,0,0,0,0, 1,0,1,1,1,1,1, 1,0,0,0,1,1,1, 0,1,1,1,1,1,1, 0,0,1,1,0,0,0, 1,1,1,1,0,0,0, 0,1,0,0,1,1,\dots$

$\sigma_0, \sigma_1, \sigma_2, \sigma_3, \sigma_3, \sigma_4, \sigma_4, \sigma_4,$

• $\sigma_5, \sigma_6, \sigma_0, \sigma_0, \sigma_1, \sigma_1, \sigma_1,$

• $\sigma_2, \sigma_3, \sigma_4, \sigma_4, \sigma_5, \sigma_5, \sigma_5,$

• $\sigma_6, \sigma_0, \sigma_1, \sigma_1, \sigma_2, \sigma_2, \sigma_2,$

• $\sigma_0, \sigma_1, \sigma_2, \sigma_2, \sigma_3, \sigma_3, \sigma_3,$

• $\sigma_4, \sigma_5, \sigma_6, \sigma_6, \sigma_0, \sigma_0, \dots$

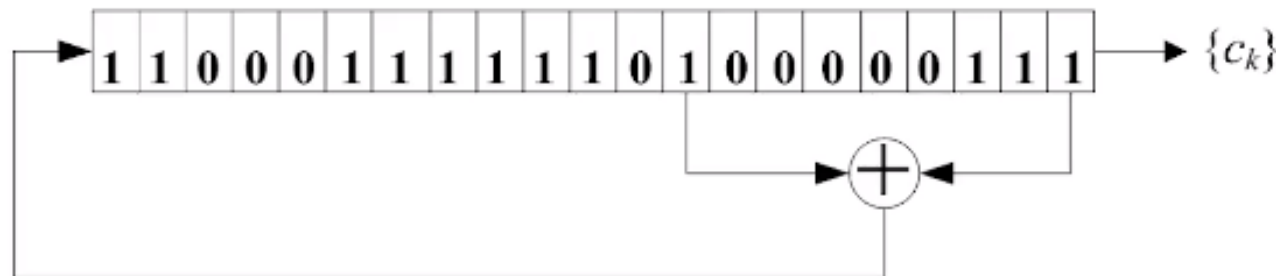
状态 (b_3, b_2, b_1)	输出
σ_0 1 1 1	1
σ_1 0 1 1	1
σ_2 0 0 1	1
σ_3 1 0 0	0
σ_4 0 1 0	0
σ_5 1 0 1	1
σ_6 1 1 0	0

2022/4/9



5、钟控序列生成器：示例

- $\{c_k\}$ 的极小特征多项式为 $1+x^{14}+x^{21}$ ，其线性复杂度为 $3 \cdot (2^3-1)=21$ ，下图是其线性等价生成器。



2022/4/9



第3单元 序列密码

提纲

CONTENTS

3.1 序列密码概述

3.2 序列密码基础

3.3 典型序列密码

2022/4/9



3.3.1 良好序列密码的原则

设计一个性能良好的序列密码是一项十分困难的任务。最基本的设计原则是“**密钥流生成器的不可预测性**”，它可分解为下述基本原则：

- ① 长周期。
- ② 高线性复杂度。
- ③ 统计性能良好。
- ④ 足够的“混乱”。
- ⑤ 足够的“扩散”。
- ⑥ 抵抗不同形式的攻击。

王华

3.3 典型序列密码



3.3.2 A5/1序列密码算法

比较常用的序列密码是A5和RC4序列密码算法。

A5是典型的基于LFSR的序列密码算法。

RC4基于分组密码的输出反馈模式（OFB）和密码反馈模式（CFB）来实现的。

其他不基于LFSR的序列密码生成器的安全性基于数论问题的难解性，这些生成器比基于LFSR的生成器要慢很多。

王华



1、A5/1序列密码算法的用法

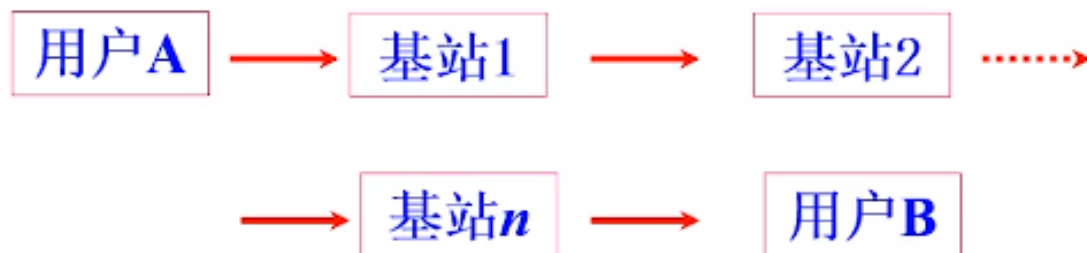
A5序列密码算法是1989年由法国人开发，用于蜂窝式电话系统（GSM）加密算法。A5-1算法用于用户的手机到基站之间的通信加密。



1、A5/1序列密码算法的用法

- 用于蜂窝式移动电话系统语音和数字加密。
- A5/1算法用于用户的手机到基站之间的通信加密，通信内容到基站后先解密变成明文，然后再进行基站到基站之间、以及基站到用户手机之间的信息加密，完成通信内容在通信过程的加密保护

通信模式



算法应用于用户A和基站1通讯的过程。
接收方用户B对消息的加脱密与用户A和基站1的通信完全类似。中间的消息传递是基站之间加密完成的。



1、A5/1序列密码算法的用法

- 应用环节
 - 只需考察用户A到基站1之间通信内容的加解密，中间消息的传送由基站到基站之间的加密完成，而接收方用户B对消息的加解密与用户A到基站1之间的通信完全类似，只不过是用户B先解密消息。
- 基本密钥 K_{A1}
 - 基本密钥 K_{A1} ：预置在SIM卡中，与基站1共享。
 - 生存期：一旦植入SIM卡将不再改变。
 - 用途：用来分配用户和基站之间的会话密钥。



1、A5/1序列密码算法的用法

- 会话密钥 k
 - 产生方式：在每次会话时，基站产生一个64比特的随机数 k 。
 - 分配方式：利用基本密钥 K_{A1} ，使用其它密码算法将 k 加密传给用户手机。
 - 生存期：仅用于一次通话时间。
- 明文处理
 - 按每帧228比特分为若干帧后逐帧加密，每帧处理方式相同。



1、A5/1序列密码算法的用法

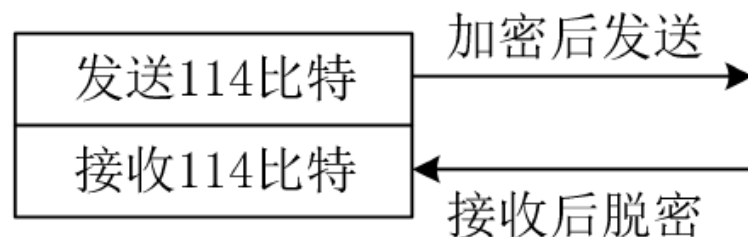
• 加密方式

- 加密： $E_k(M) = E_{k1}(M_1)E_{k2}(M_2)E_{k3}(M_3)\dots$
- 一次通话使用一个会话密钥，对每帧使用不同的帧密钥
- 帧会话密钥：帧序号，长度为**22**比特
- 帧会话密钥共产生**228**比特密钥流，实现对本帧**228**比特通信数据的加解密
- 明密结合方式：逐位异或
- 一次通话量：至多 **2^{22}** 帧数据，约 **0.89×2^{30}** 比特

A5算法中，明文按照每帧**228**比特分为若干帧后逐帧加密

$$M = M_1 \parallel M_2 \parallel \dots \parallel M_i \parallel \dots \quad |M_i| = 228$$

每帧处理方式如下：





2、A5/1序列密码算法的LFSR

- 算法使用3个级数为19、22和23的本原移存器。

$$\text{LFSR-1} \longrightarrow f_1(x) = x^{19} \oplus x^{18} \oplus x^{17} \oplus x^{14} \oplus 1$$

$$\text{LFSR-2} \longrightarrow f_2(x) = x^{22} \oplus x^{21} \oplus x^{17} \oplus x^{13} \oplus 1$$

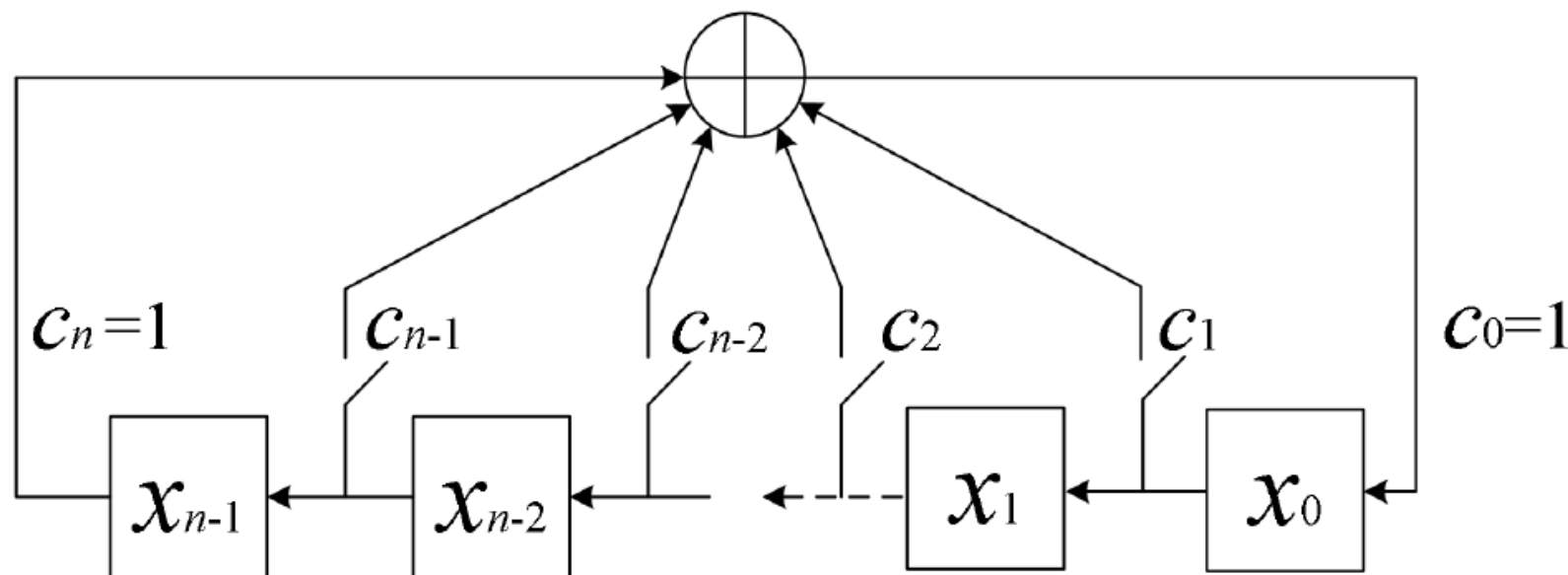
$$\text{LFSR-3} \longrightarrow f_3(x) = x^{23} \oplus x^{22} \oplus x^{19} \oplus x^{18} \oplus 1$$

三个寄存器共有64 bits。

注：A5/1算法中，LFSR的移位方式是左移方式。各寄存器的编号从第0级编号到第*n*-1级。

2、A5/1序列密码算法的LFSR

n 级左移LFSR的结构框图

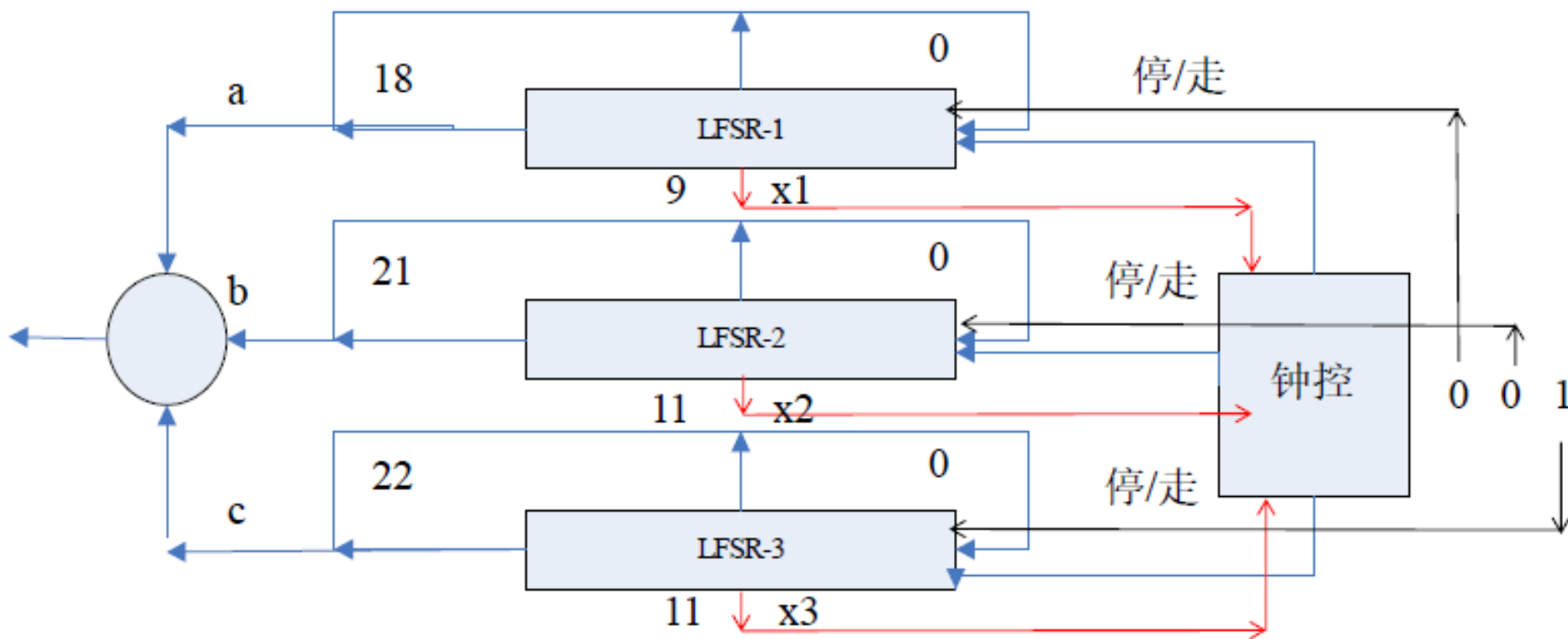


移寄存器的左移和右移方式，除移位方式不同外，其工作原理完全相同。



2、A5/1序列密码算法的LFSR

A5-1算法的结构框图



前馈函数 $f(x_1, x_2, x_3) = x_1 \oplus x_2 \oplus x_3$



2、A5/1序列密码算法的LFSR:初始化

- 初始化是利用一次通话的会话密钥 k 和帧序号设定三个移寄存器的起点，即初始状态。
- **Step 1:** 将三个LFSR的初态都设置为全零向量；
- **Step 2:** (密钥参与) 三个LFSR都规则动作64次，每次动作1步。
- 在第 i 步动作时，三个LFSR的反馈内容都首先与密钥的第 i 比特异或，并将异或结果作为LFSR反馈的内容。



2、A5/1序列密码算法的LFSR：密钥

密钥K采用64 bits。初始时密钥K被载入3个寄存器。

以寄存器1为例的密钥参与过程：

$$k = k_{64}k_{63} \cdots k_1 \quad f_1(x) = x^{19} \oplus x^{18} \oplus x^{17} \oplus x^{14} \oplus 1$$

初始状态： $S_0 = (x_{18}, x_{17}, \cdots, x_0) = (0, 0, \cdots, 0)$

动作1步后状态： $S_1 = (x_{18}, x_{17}, \cdots, x_0) = (0, 0, \cdots, k_1)$

动作2步后状态： $S_2 = (x_{18}, x_{17}, \cdots, x_0) = (0, 0, \cdots, k_1, k_2)$

动作14步后状态： $S_{14} = (x_{18}, x_{17}, \cdots, x_0) = (0, 0, \cdots, k_1, k_2, \cdots, k_{13}, k_{14})$

动作15步后状态： $S_{15} = (x_{18}, x_{17}, \cdots, x_0) = (0, 0, \cdots, k_1, k_2, \cdots, k_{14}, k_1 \oplus k_{15})$

动作64步完成密钥参与过程。



2、A5/1序列密码算法的LFSR：帧

- **Step 3:** (帧序号参与) 三个LFSR都规则动作22次，每次动作1步。在第*i*步动作时，三个LFSR的反馈内容都首先与帧序号的第*i*比特异或，并将异或的结果作为LFSR反馈的内容；帧序号比特的序号是从最低位编到最高位。
- 帧序号参与方式：与密钥参与方式相同，不同的明文数据帧按顺序编号，每个编号为22比特。
- 记帧序号为

$$T_0 = t_{22}t_{21} \cdots t_1 = 00 \cdots 00 \quad T_1 = t_{22}t_{21} \cdots t_1 = 00 \cdots 01 \quad \dots$$

- 帧密钥参与的目的：对不同的帧设置不同的帧会话密钥，保证对每帧以不同的起点生成密钥流，尽可能避免密钥重用。



2、A5/1序列密码算法的LFSR：帧

- 对每一步做: $m = \text{major}(x_8, y_{10}, z_{10})$
 - Major (多数) 函数定义: $\text{major}(0,1,0) = 0$ and $\text{major}(1,1,0) = 1$
- 如果 $x_8 = m$ 那么 X寄存器 进行移位运算
 - $t = x_{13} \oplus x_{16} \oplus x_{17} \oplus x_{18}$
 - $x_i = x_{i-1}$ for $i = 18, 17, \dots, 1$ and $x_0 = t$
- 如果 $y_{10} = m$ 那么 Y寄存器进行移位运算
 - $t = y_{20} \oplus y_{21}$
 - $y_i = y_{i-1}$ for $i = 21, 20, \dots, 1$ and $y_0 = t$
- 如果 $z_{10} = m$ 那么 Z寄存器进行移位运算
 - $t = z_7 \oplus z_{20} \oplus z_{21} \oplus z_{22}$
 - $z_i = z_{i-1}$ for $i = 22, 21, \dots, 1$ and $z_0 = t$
- 密匙流比特最后由 $x_{18} \oplus y_{21} \oplus z_{22}$ 产生

王华



3、A5/1序列密码算法的密钥流：生成

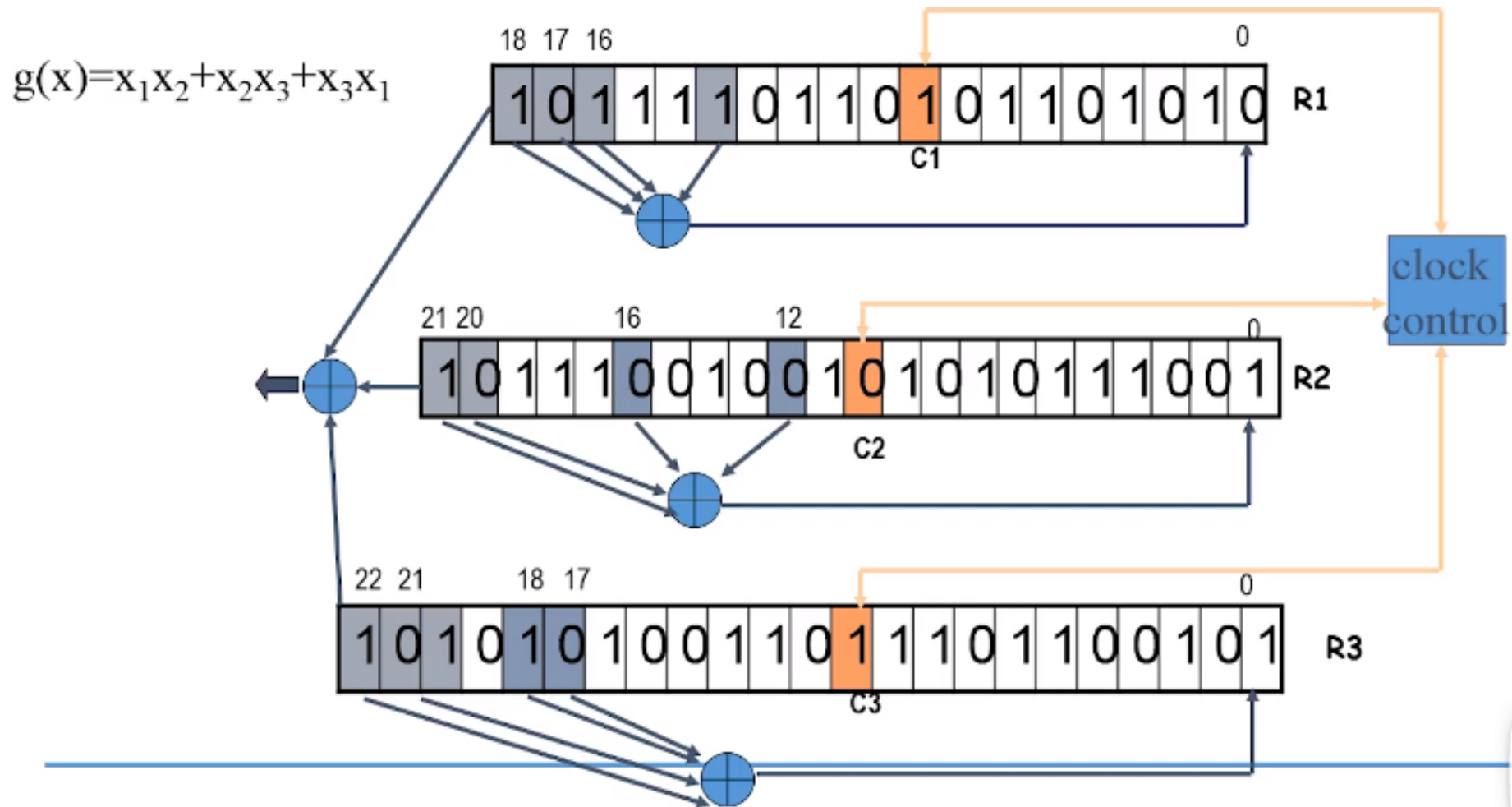
- A5算法中，采用钟控方式控制3个LFSR来产生密钥流。
- 钟控信号 $x_1 x_2 x_3$ 的采取： x_1 取自LFSR-1第9级； x_2 取自LFSR-2第11级； x_3 取自LFSR-3第11级。
- 控制方式：采用多项式 $g(x)=x_1x_2+x_2x_3+x_3x_1$ 来确定，取钟控信号和多项式值相同的进行移位，不同的就不动。

(X_1, X_2, X_3)	000	001	010	011	100	101	110	111
LFSR-1	动	动	动	不动	不动	动	动	动
LFSR-2	动	动	不动	动	动	不动	动	动
LFSR-3	动	不动	动	动	动	动	不动	动

控制方式：择多原则。即0比1多时，0动1不动。1比0多，则1动0不动。

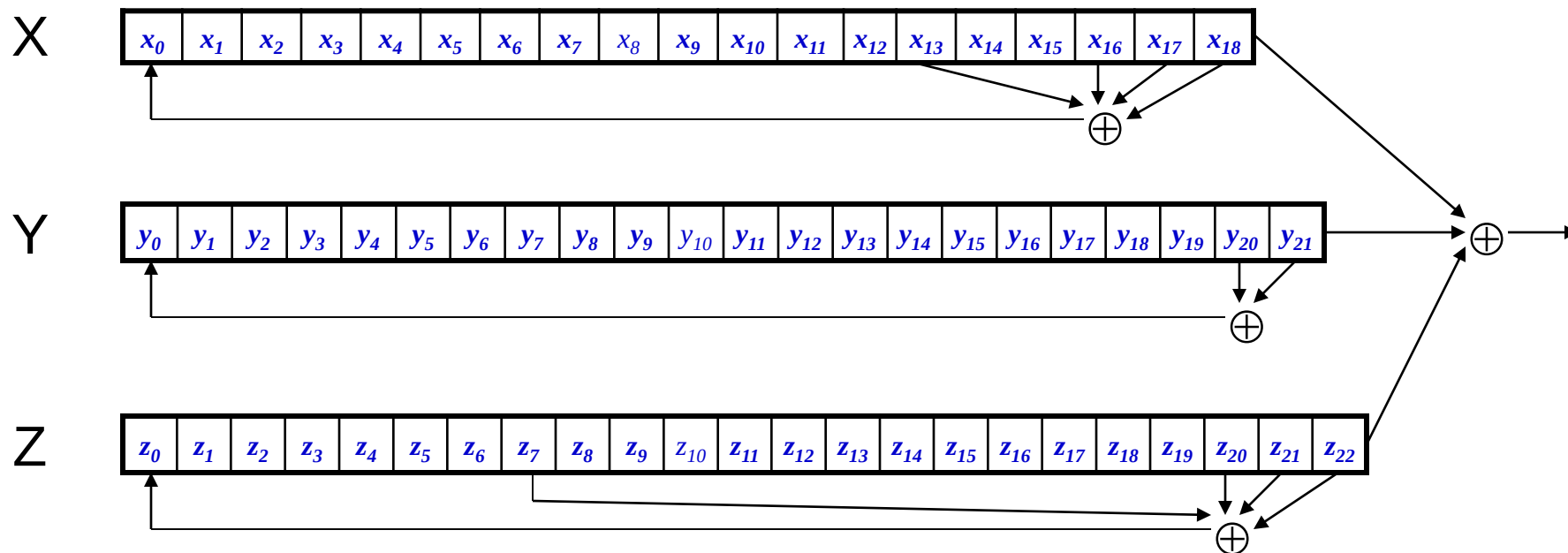


3、A5/1序列密码算法的密钥流：钟控





3、A5/1序列密码算法的密钥流：钟控



- 每次运算获得一个比特
- 密匙用于初始化三个寄存器
- 每个寄存器是否进行移位操作由 $M(x_8, y_{10}, z_{10})$ 决定
- 密匙流比特由三个最右端比特进行XOR运算获得

王立华



3、A5/1序列密码算法的密钥流：加密

- **Step 4:** 三个LFSR以钟控方式连续动作100次，但不输出密钥流；
- **Step 5:** 三个LFSR以钟控方式连续动作114次，在每次动作后，三个LFSR都将最高位寄存器中的值输出，这三个比特的异或就是当前时刻输出的1比特密钥。
- 连续动作114步，共输出114比特密钥流，用于对用户手机到基站传送的114比特数据的加密；

加密方式： $c_i = m_i \oplus d_i; i = 1, 2, \dots, 114$



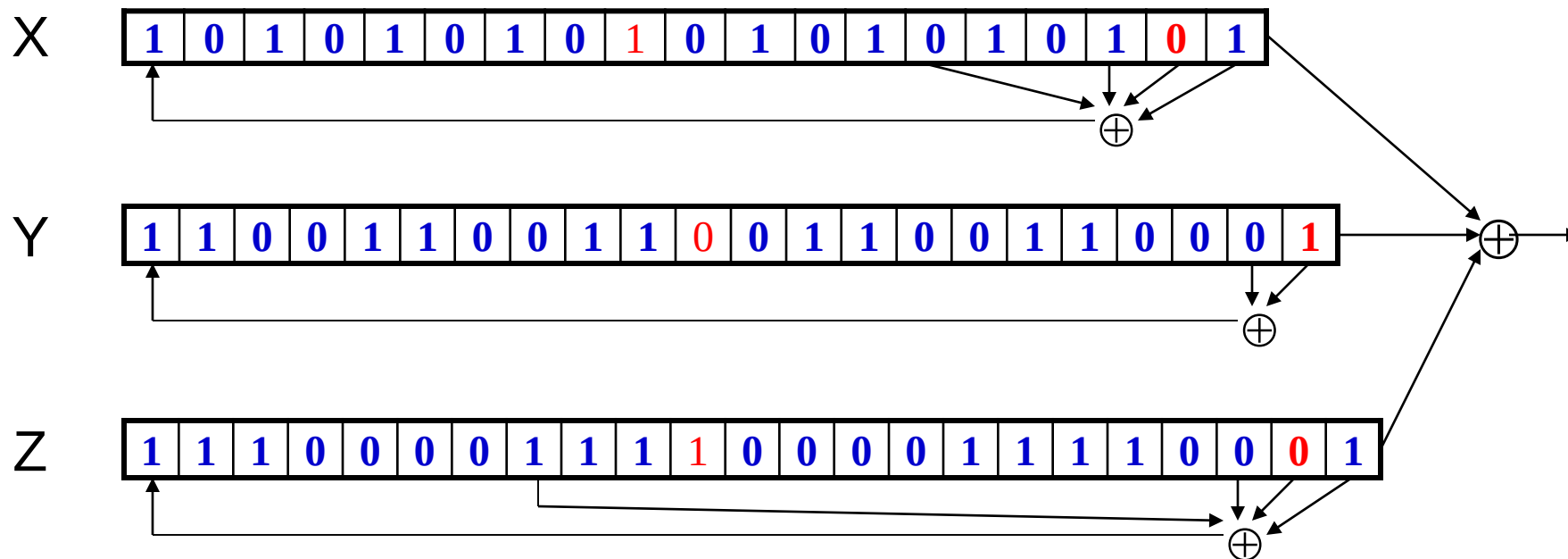
3、A5/1序列密码算法的密钥流：解密

- **Step 6:** 三个LFSR以钟控方式连续动作100次，但不输出密钥流；
- **Step 7:** 三个LFSR以钟控方式连续动作114次，在每次动作后，三个LFSR都将最高级寄存器中的值输出，这三个比特的模2和就是当前时刻输出的1比特密钥流。
- 连续动作114步，共输出114比特密钥流，这114比特用于对基站到用户手机传送的114比特数据的解密。

解密方式： $m'_i = c'_i \oplus d_i; i = 115, 116, \dots, 228$



4、A5/1序列密码算法示例



- 在这种情况下, $m = \text{maj}(x_8, y_{10}, z_{10}) = \text{maj}(1, 0, 1) = 1$
- 寄存器 X 进行移位, Y 不进行移位, Z 进行移位
- 密匙流比特由最右端比特进行XOR操作而得
- 此例, 密匙流比特是 $0 \oplus 1 \oplus 0 = 1$
- 最后用密匙流和明文做XOR运算进行加密和解密

王华



5、A5/1序列密码算法的弱点

- 移位寄存器太短容易遭受穷举攻击
 - A5/1算法把主密钥作为算法中三个寄存器的初始值，长度为64比特。如果利用已知明文攻击，只需要知道其中两个寄存器的初始值，就可以计算出另一个寄存器的初始值，这只需要 2^{40} 步就可以得出寄存器LFSR-1和LFSR-2的结构。
- 事实上，A5/1加密算法中存在严重的安全问题
 - 利用了GSM通信加密中的两个安全漏洞，可以通过离线迭代计算生成一个彩虹表，它包含有密钥和其相对应的输出密码。这个彩虹表的大小为984GB。得到了彩虹表之后，安全专家就可以在短短的九秒内确定用于加密通信数据的密钥。



6、A5/1序列密码算法的思考

A5的密钥长度是 [填空1]，采用的设计结构是 [填空2]，特点是便于 [填空3] 实现，速度 [填空4]。

3.3 典型序列密码



3.3.3 RC4算法

RC4算法——密钥生成算法

8×8的S盒： $S_0, S_1, \dots, S_{255}$ ，所有项是数字0-255的置换。两个指针（计数器） i 、 j 的初值为0。

$$i = (i + 1) \bmod 256$$

$$j = (j + S_i) \bmod 256$$

交换 S_i 和 S_j

$$t = (S_i + S_j) \bmod 256$$

$$K = S_t$$

加密方式： $m \oplus K = c$ （以字节为单位，按位异或）

2022/4/9



1、RC4:应用

RC4目前使用在:

- SSL（安全套接字）中广泛使用
- WEP (Wired Equivalent Privacy: 有线对等保密)
IEEE802.11

(http://biz.chinabyte.com/159/2027659_1.shtml)

2022/4/9

3.3.4 其他序列密码算法

●ECRYPT在序列密码研究方面开展了一系列的活动。

- 2004年10月14 ~ 15日，在比利时举行了一个名为SASC的特别会议，会议的讨论引发了序列密码算法的征集活动，建立序列密码工程（eStream）。
- 2004年11月发布了征集公告，这也是对NESSIE没有征集到序列密码算法的一个补充。
- 征集活动到2005年4月29日结束，根据四个征集原则一共征集到了34个序列密码算法，Salsa, Trivium, Dragon都在其中。

王卫华



1、ECRYPT:序列密码

- 征集到的流密码算法是
 - ABC、Achterbahn、CryptMT、DECIM、DICING、DRAGON、Edon80、F-FCSR、Frogbit、Grain、HC.256、HermesS、LEX、MAG、MICKEY、MICKEY-128、Mir·I、MOSQUITO、NLS、Phdix、Polar Bear、POMARANCH、Py、Rabbit > Salsa20、SFINKS、SOSEMANUK、SSS、TRBDK3YAEA、Trivium、TSO3、VEST、WG、Yamb 和ZIGCrypt.

2022/4/9



1、ECRYPT:序列密码

- 算法设计思路可分为：
 - 基于线性反馈移位寄存器(LFSR)的设计
 - 基于非线性反馈移位寄存器(NFSR)的设计
 - 基于表驱动的设计
 - 基于分组密码部件的设计
- 计划**2006年2月**结束第一阶段的评估，第二阶段在**2006年6月到2007年7月**进行，**2008年1月**完成评审活动。

王卫华

2、Trivium :结构图



- 结构图

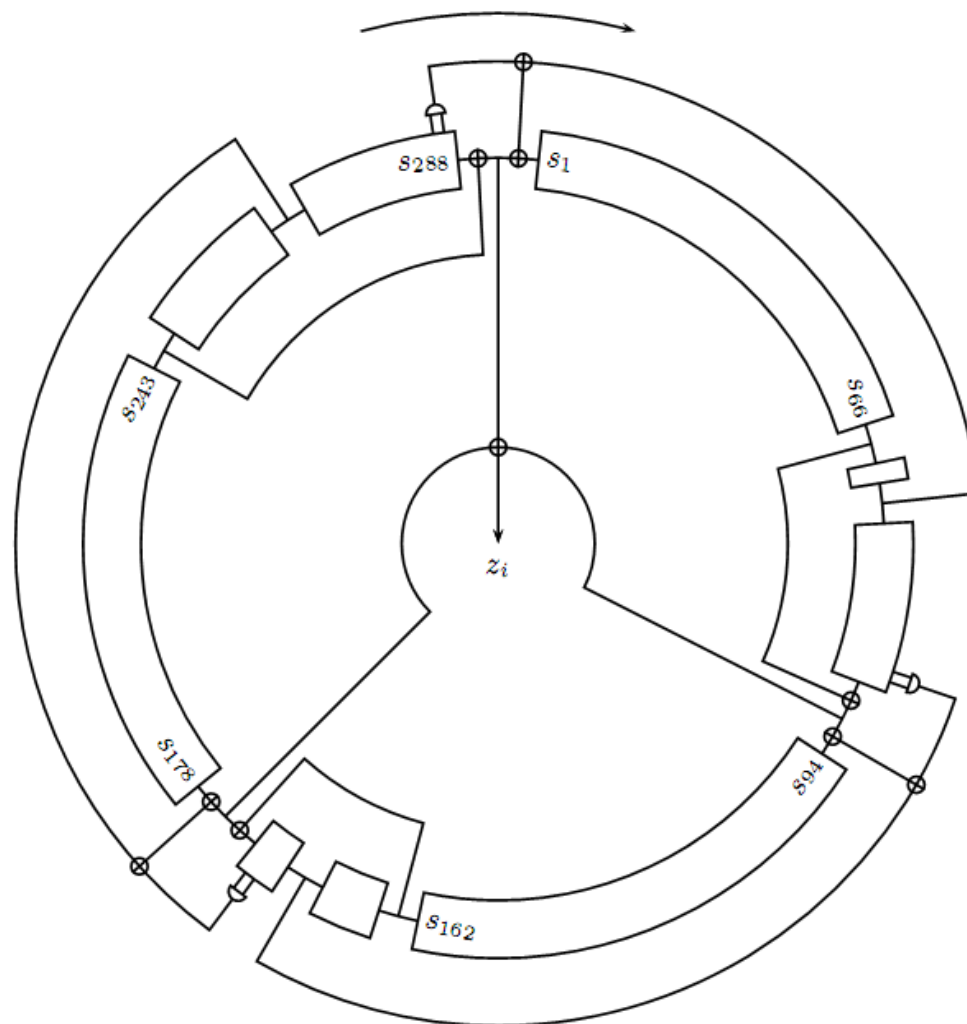


Fig. 1. TRIVIUM



2、Trivium :初始化

- 初始状态填充:

$$(s_1, s_2, \dots, s_{93}) \leftarrow (K_1, \dots, K_{80}, 0, \dots, 0)$$

$$(s_{94}, s_{95}, \dots, s_{177}) \leftarrow (IV_1, \dots, IV_{80}, 0, \dots, 0)$$

$$(s_{178}, s_{279}, \dots, s_{288}) \leftarrow (0, \dots, 0, 1, 1, 1)$$



2、Trivium :算法

for $i = 1$ **to** N **do**

$$t_1 \leftarrow s_{66} + s_{93}$$

$$t_2 \leftarrow s_{162} + s_{177}$$

$$t_3 \leftarrow s_{243} + s_{288}$$

$$z_i \leftarrow t_1 + t_2 + t_3$$

$$t_1 \leftarrow t_1 + s_{91} \cdot s_{92} + s_{171}$$

$$t_2 \leftarrow t_2 + s_{175} \cdot s_{176} + s_{264}$$

$$t_3 \leftarrow t_3 + s_{286} \cdot s_{287} + s_{69}$$

$$(s_1, s_2, \dots, s_{93}) \leftarrow (t_3, s_1, \dots, s_{92})$$

$$(s_{94}, s_{95}, \dots, s_{177}) \leftarrow (t_1, s_{94}, \dots, s_{176})$$

$$(s_{178}, s_{279}, \dots, s_{288}) \leftarrow (t_2, s_{178}, \dots, s_{287})$$

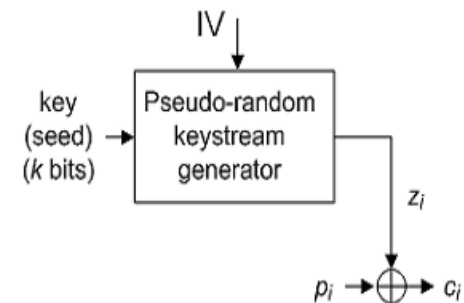
end for

Parameters

Key size: 80 bit

IV size: 80 bit

Internal state: 288 bit



2024



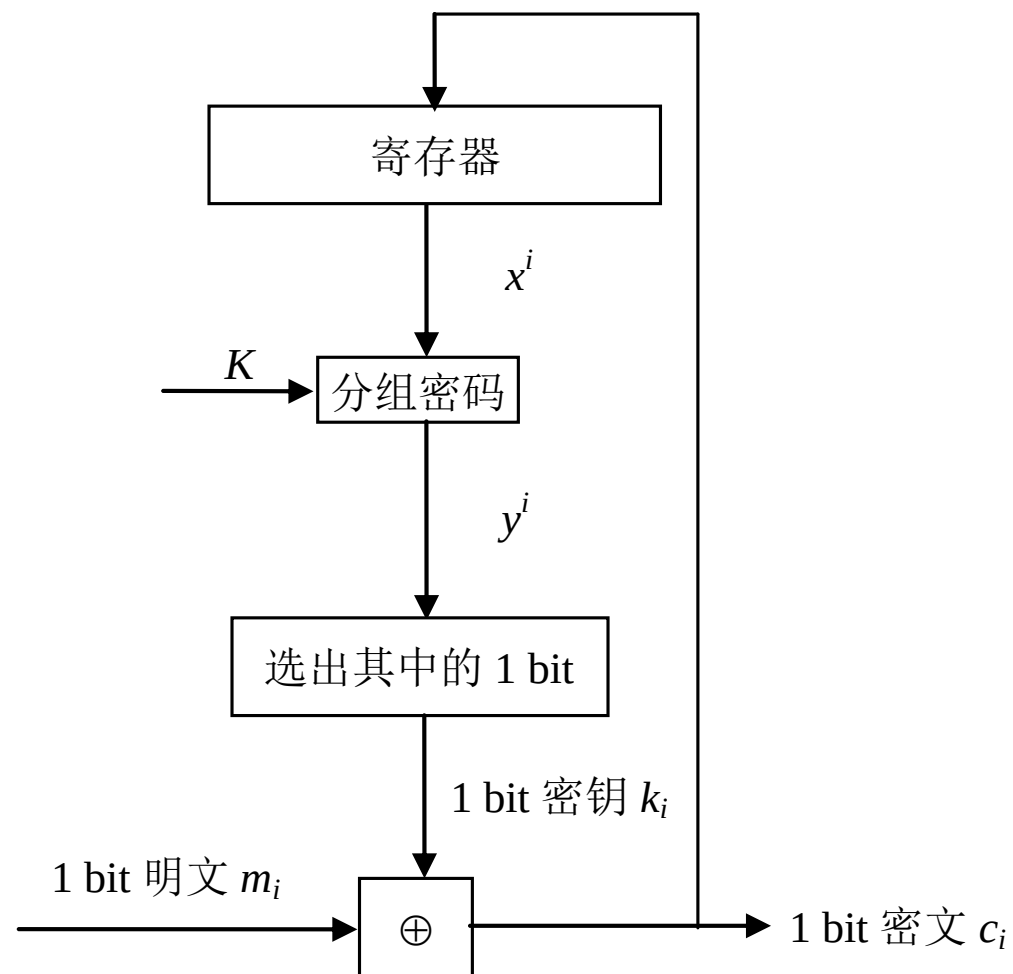
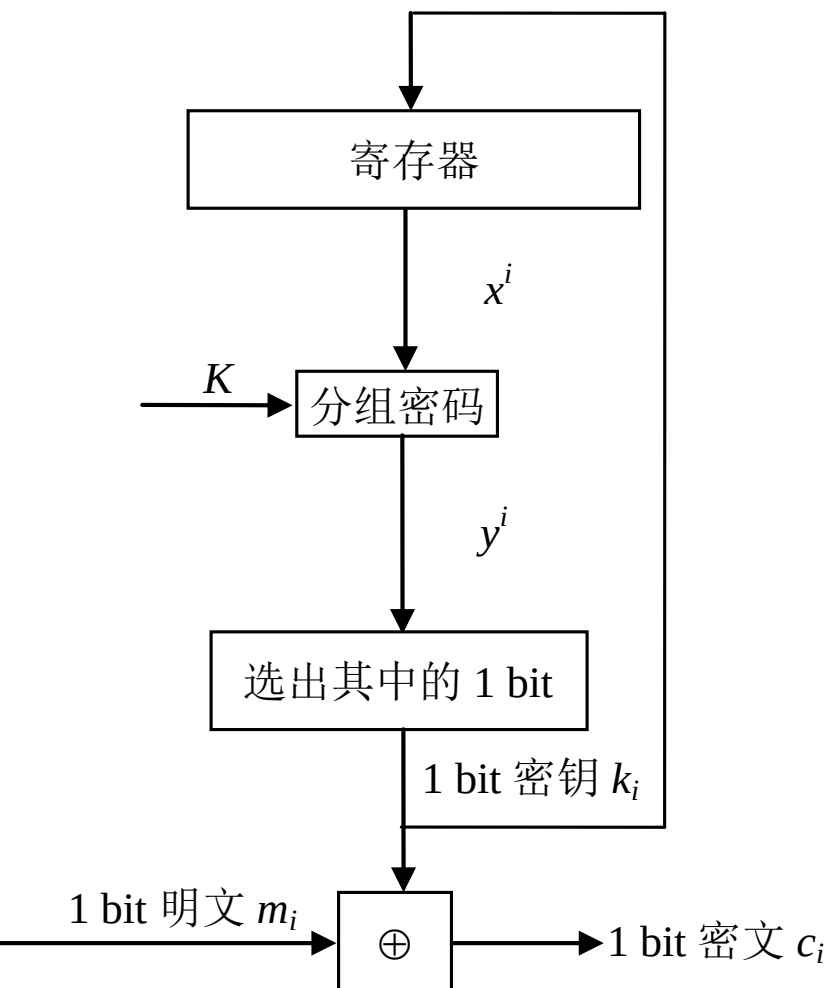
2、Trivium :思考

1. trivium的80位密钥长度可否增加？能否给出一种增加方式？
2. 如果量子计算机出现，则通过搜索算法可将密钥长度减半。能否简单改变算法，使之有更长的密钥？

2024



3、基于分组密码的序列密码



2024



4、Salsa20 :

$$Salsa20_k(IV, i) = H \begin{pmatrix} c_0 & k_0 & k_1 & k_2 \\ k_3 & c_1 & v_0 & v_1 \\ i_0 & i_1 & c_2 & k_4 \\ k_5 & k_6 & k_7 & c_3 \end{pmatrix}$$

where

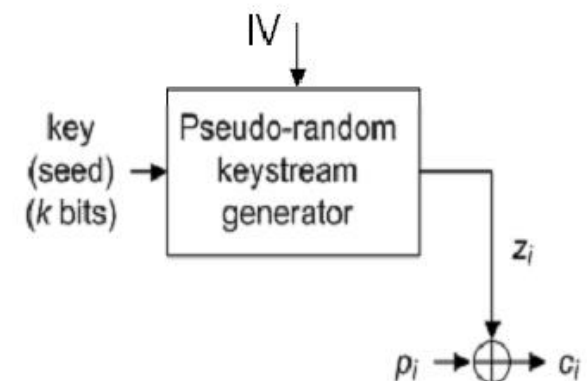
$$H(x) = x + R^{12}(x)$$

$k = k_0, k_1, \dots, k_7$ a 256-bit secret key,

$IV = v_0, v_1$ a 64-bit initial vector.

$i = i_0, i_1$ a 64-bit counter,

$c = c_0, c_1, c_2, c_3$ 128-bit constants.



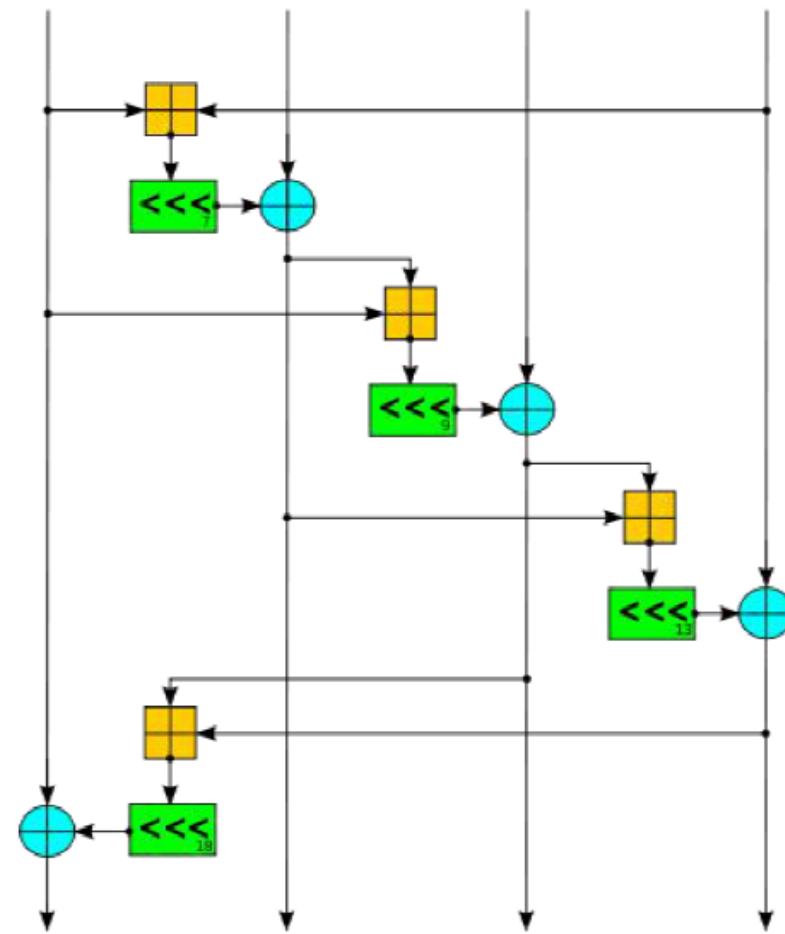
2024



4、Salsa20 :Q变换模块

- 输入为4个字

$$\begin{aligned} z_1 &= y_1 \oplus ((y_0 + y_3) \lll 7), \\ z_2 &= y_2 \oplus ((z_1 + y_0) \lll 9), \\ z_3 &= y_3 \oplus ((z_2 + z_1) \lll 13), \\ z_0 &= y_0 \oplus ((z_3 + z_2) \lll 18). \end{aligned}$$



2014

4、Salsa20 :轮函数



4×4 word matrix x ,

$$x = \begin{pmatrix} x_0 & x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 & x_7 \\ x_8 & x_9 & x_{10} & x_{11} \\ x_{12} & x_{13} & x_{14} & x_{15} \end{pmatrix}$$

The round function $R(x)$ is defined as follows,

$$R(x) = (Q'^4(x))^T,$$

where

$$Q'(x) = \begin{pmatrix} x_5 & x_6 & x_7 & q_1 \\ x_9 & x_{10} & x_{11} & q_2 \\ x_{13} & x_{14} & x_{15} & q_3 \\ x_1 & x_2 & x_3 & q_0 \end{pmatrix}$$

and

$$\begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{pmatrix} = Q \begin{pmatrix} x_0 \\ x_4 \\ x_8 \\ x_{12} \end{pmatrix}.$$

4、Salsa20 :思考



郑州大学
Zhengzhou University

salsa的密钥长度是多少 [填空1]，它的计数器有什么作用 [填空2]，它的初始向量有什么作用 [填空3]，它的设计思路是 [填空4]。

王华



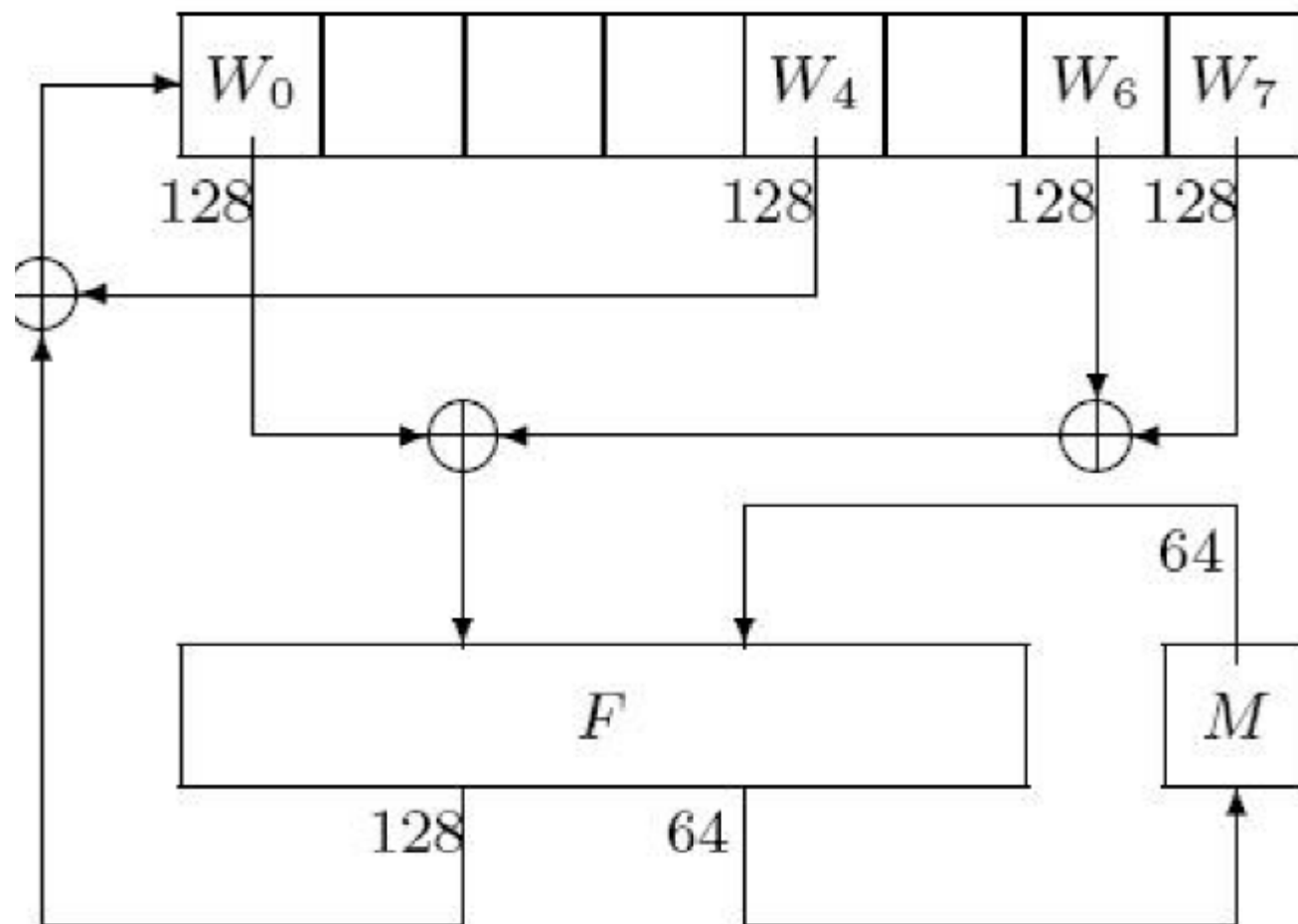
5、Dragon:简介

- Dragon是澳大利亚的K.Chen等提交的面向字的序列密码。
- Dragon序列密码的设计融合了分组密码的设计思想， 可以看作是分组密码的输出反馈模式(OFB)构成的序列密码。
- 它包含了一个1024比特的NFSR， 一个状态更新函数和输出计数器 M 。

2024



5、Dragon: 密钥安装结构初始化



2024



5、Dragon:密钥安装结构初始化

Dragon 有一种使用密钥和公开的初始化向量来产生密钥或密钥重用的简单策略。1024 位的内部状态被分为 8 个 128 位的字，依次记为 W_0 到 W_7 。内部状态最初由密钥和初始化向量填充。状态的初始化过程多次利用 F 函数——包含 F 函数的 16 次反复。64 位存贮器 $M = 0x0000447261676F6E$ 。

2024



5、Dragon:密钥安装结构初始化

- Dragon有两个不同的密钥和初始化变量，一个是128位,一个是256位。标记256位的密钥和初始化变量为K和IV，标记128位的密钥和初始化变量为k和iv。
- Dragon每产生 2^{64} 位密钥流就重置一次密钥。

输入= $\{K, IV\}$ (256-bit) 输入= $\{k, iv\}$ (128-bit)

1、 $W_0 \parallel \dots \parallel W_7 = K \parallel K \oplus IV \parallel \overline{K \oplus IV} \parallel IV$ (256 bit)

$W_0 \parallel \dots \parallel W_7 = k \parallel k' \oplus iv' \parallel iv \parallel k \oplus iv' \parallel k' \parallel k \oplus iv \parallel iv' \parallel k' \oplus iv$ (128-bit)

2、 $M = 0x0000447261676F6E$

2024



5、Dragon:密钥初始化过程

执行步骤 3~8, 16 次:

3、 $a \parallel b \parallel c \parallel d = (W_0 \oplus W_6 \oplus W_7)$

4、 $e \parallel f = M$

5、 $\{a', b', c', d', e', f'\} = F(a, b, c, d, e, f)$

6、 $W_0 = (a' \parallel b' \parallel c' \parallel d') \oplus W_4$

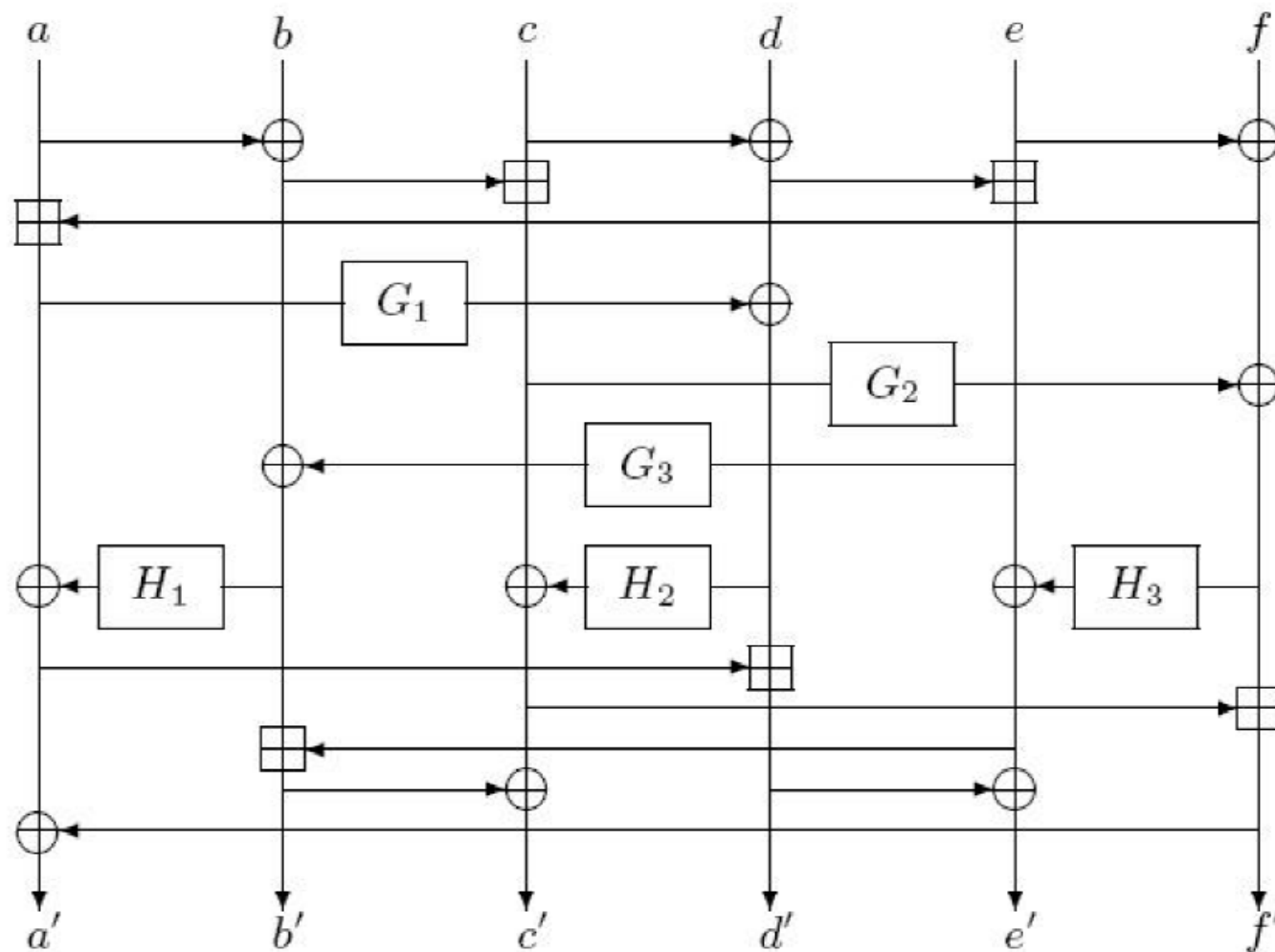
7、 $W_i = W_{i-1}, i=7,6, \dots, 1$

8、 $M = e' \parallel f'$

输出 $= \{W_0 \parallel \dots \parallel W_7\}$

2024

5、Dragon:F结构图



2024



5、Dragon:F结构图解析

- $\oplus$ 表示异或运算， $\boxplus$ 表示模 2^{32} 的加法。

输入 = $\{a, b, c, d, e, f\}$

前混合层:

$$1、b = b \oplus a; \quad d = d \oplus c; \quad f = f \oplus e;$$

$$2、c = c \boxplus b; \quad e = e \boxplus d; \quad a = a \boxplus f;$$

S 盒层:

$$3、d = d \oplus G_1(a); \quad f = f \oplus G_2(c); \quad b = b \oplus G_3(e);$$

$$4、a = a \oplus H_1(b); \quad c = c \oplus H_2(d); \quad e = e \oplus H_3(f);$$

后混合层:

$$5、d' = d \boxplus a; \quad f' = f \boxplus c; \quad b' = b \boxplus e;$$

$$6、c' = c \oplus b; \quad e' = e \oplus d; \quad a' = a \oplus f;$$

输出 = $\{a', b', c', d', e', f'\}$

2024



5、Dragon:内部函数

- 32位的 x 分成4个8比特 x_0, x_1, x_2, x_3 ，每8比特查一次表（16行16列，共256个输出值）

$$G_1(x) = S_1(x_0) \oplus S_1(x_1) \oplus S_1(x_2) \oplus S_2(x_3)$$

$$G_2(x) = S_1(x_0) \oplus S_1(x_1) \oplus S_2(x_2) \oplus S_1(x_3)$$

$$G_3(x) = S_1(x_0) \oplus S_2(x_1) \oplus S_1(x_2) \oplus S_1(x_3)$$

$$H_1(x) = S_2(x_0) \oplus S_2(x_1) \oplus S_2(x_2) \oplus S_1(x_3)$$

$$H_2(x) = S_2(x_0) \oplus S_2(x_1) \oplus S_1(x_2) \oplus S_2(x_3)$$

$$H_3(x) = S_2(x_0) \oplus S_1(x_1) \oplus S_2(x_2) \oplus S_2(x_3)$$

2024

5、Dragon:S1

sbox1[256]={

0x393BCE6B,0x232BA00D,0x84E18ADA,0x84557BA7,0x56828948,0x166908F3,
0x414A3437,0x7BB44897,0x2315BE89,0x7A01F224,0x7056AA5D,0x121A3917,
0xE3F47FA2,0x1F99D0AD,0x9BAD518B,0x99B9E75F,0x8829A7ED,0x2C511CA9,
0x1D89BF75,0xF2F8CDD0,0x2DA2C498,0x48314C42,0x922D9AF6,0xAA6CE00C,
0xAC66E078,0x7D4CBOCO,0x5500C6E8,0x23E4576B,0x6B365D40,0xEE171139,
0x336BE860,0x5DBEEFE,0x0E945776,0xD4D52CC4,0x0E9BB490,0x376EB6FD,
0x6D891655,0xD4078FEE,0xE07401E7,0xA1E4350C,0xABC78246,0x73409C02,
0x24704A1F,0x478ABB2C,0xA0849634,0x9E9E5FEB,0x77363D8D,0xD350BC21,
0x876E1BB5,0xC8F55C9D,0xD112F39F,0xDF1A0245,0x9711B3F0,0xA3534F64,
0x42FB629E,0x15EAD26A,0xD1CFA296,0x7B445FEE,0x88C28D4A,0xCA6A8992,
0xB40726AB,0x508C65BC,0xBE87B3B9,0x4A894942,0x9AECC5B,0x6CA6F10B,
0x303F8934,0xD7A8693A,0x7C8A16E4,0xB8CF0AC9,0xAD14B784,0x819FF9F0,
0xF20DCDFA,0xB7CB7159,0x58F3199F,0x9855E43B,0x1DF6C2D6,0x46114185,
0xE46F5D0F,0xAAC70B5B,0x48590537,0x0FD77B28,0x67D16C70,0x75AE53F4,
0xF7BFEC A1,0x6017B2D2,0xD8A0FA28,0xB8FC2E0D,0x80168E15,0x0D7DEC9D,
0xC5581F55,0xBE4A2783,0xD27012FE,0x53EA81CA,0xEBAA07D2,0x54F5D41D,
0xABB26FA6,0x41B9EAD9,0xA48174C7,0x1F3026F0,0xEFBADD8E,0x387E9014,
0x1505AB79,0xEADF0DF7,0x67755401,0xDA2EF962,0x41670B0E,0x0E8642F2,
0xCE486070,0xA47D3312,0x4D7343A7,0xECDA58D0,0x1F79D536,0xD362576B,
0x9D3A6023,0xC795A610,0xAE4DF639,0x60C0B14E,0xC6DD8E02,0xBDE93F4E,
0xB7C3B0FF,0x2BE6BCAD,0xE4B3FDFD,0x79897325,0x3038798B,0x08AE6353,
0x7D1D20EB,0x3B208D21,0xD0D6D104,0xC5244327,0x9893F59F,0xE976832A,
0xB1EB320B,0xA409D915,0x7EC6B543,0x66E54F98,0x5FF805DC,0x599B223F,
0xAD78B682,0x2CF5C6E8,0x4FC71D63,0x08F8FED1,0x81C3C49A,0xE4D0A778,
0xB5D369CC,0x2DA336BE,0x76BC87CB,0x957A1878,0xFA136FBA,0x8F3C0E7B,
0x7A1FF157,0x598324AE,0xFFBAAC22,0xD67DE9E6,0x3EB52897,0x4E07E855,
0x87CE73F5,0x8D046706,0xD42D18F2,0xE71B1727,0x38473B38,0xB37B24D5,
0x381C6AE1,0xE77D6589,0x6018CBFF,0x93CF3752,0x9B6EA235,0x504A50E8,
0x464EA180,0x86AFBE5E,0xCC2D6AB0,0xAB91707B,0x1DB4D579,0xF9FAFD24,
0x2B28CC54,0xCDCFD6B3,0x68A30978,0x43A6DFD7,0xC81DD98E,0xA6C2FD31,
0x0FD07543,0xAFB400CC,0x5AF11A03,0x2647A909,0x24791387,0x5CFB4802,
0x88CE4D29,0x353F5F5E,0x7038F851,0xF1F1C0AF,0x78EC6335,0xF2201AD1,
0xDF403561,0x4462DFC7,0xE22C5044,0x9C829EA3,0x43FD6EAE,0x7A42B3A7,
0x5BFAAAEC,0x3E046853,0x5789D266,0xE1219370,0xB2C420F8,0x3218BD4E,
0x84590D94,0xD51D3A8C,0xA3AB3D24,0x2A339E3D,0xFEE67A23,0xAF844391,
0x17465609,0xA99AD0A1,0x05CA597B,0x6024A656,0x0BF05203,0x8F559DDC,
0x894A1911,0x909F21B4,0x6A7B63CE,0xE28DD7E7,0x4178AA3D,0x4346A7AA,
0xA1845E4C,0x166735F4,0x639CA159,0x58940419,0x4E4F177A,0xD17959B2,
0x12AA6FFD,0x1D39A8BE,0x7667F5AC,0xED0CE165,0xF1658FD8,0x28B04E02,
0x1FA480CF,0xD3FB6FEF,0xED336CCB,0x9EE3CA39,0x9F224202,0x2D12D6E8,
0xFAAC50CE,0xFA1E98AE,0x61498532,0x03678CC0,0x9E85EFD7,0x3069CE1A,
0xF115D008,0x4553AA9F,0x3194BE09,0xB4A9367D,0x0A9DFEEC,0x7CA002D6,
0x8E53A875,0x965E8183,0x14D79DAC,0x0192B555};

5、Dragon:S2

sbox2[256]={

```
0xA94BC384,0xF7A81CAE,0xAB84ECD4,0x00DEF340,0x8E2329B8,0x23AF3A22,
0x23C241FA,0xAED8729E,0x2E59357F,0xC3ED78AB,0x687724BB,0x7663886F,
0x1669AA35,0x5966EAC1,0xD574C543,0xDBC3F2FF,0x4DD44303,0xCD4F8D01,
0x0CBF1D6F,0xA8169D59,0x87841E00,0x3C515AD4,0x708784D6,0x13EB675F,
0x57592B96,0x07836744,0x3E721D90,0x26DAA84F,0x253A4E4D,0xE4FA37D5,
0x9C0830E4,0xD7F20466,0xD41745BD,0x1275129B,0x33D0F724,0xE234C68A,
0x4CA1F260,0x2BB0B2B6,0xBD543A87,0x4ABD3789,0x87A84A81,0x948104EB,
0xA9AAC3EA,0xBAC5B4FE,0xD4479EB6,0xC4108568,0xE144693B,0x5760C117,
0x48A9A1A6,0xA987B887,0xDF7C74E0,0xBC0682D7,0xEDB7705D,0x57BFFEEA,
0x8A0BD4F1,0x1A98D448,0xEA4615C9,0x99E0CBD6,0x780E39A3,0xADBCD406,
0x84DA1362,0x7A0E984B,0xBED853E6,0xD05D610B,0x9CAC6A28,0x1682ACDF,
0x889F605F,0x9EE2FEBA,0xDB556C92,0x86818021,0x3CC5BEA1,0x75A934C6,
0x95574478,0x31A92B9B,0xBFE3E92B,0xB28067AE,0xD862D848,0x0732A22D,
0x840EF879,0x79FFA920,0x0124C8BB,0x26C75B69,0xC3DAAAC5,0x6E71F2E9,
0x9FD4AFA6,0x474D0702,0x8B6AD73E,0xF5714E20,0xE608A352,0x2BF644F8,
0x4DF9A8BC,0xB71EAD7E,0x6335F5FB,0x0A271CE3,0xD2B552BB,0x3834A0C3,
0x341C5908,0x0674A87B,0x8C87C0F1,0xFF0842FC,0x48C46BDB,0x30826DF8,
0x8B82CE8E,0x0235C905,0xDE4844C3,0x296DF078,0xEFAA6FEA,0x6CB98D67,
0x6E959632,0xD5D3732F,0x68D95F19,0x43FC0148,0xF808C7B1,0xD45DBD5D,
0x5DD1B83B,0x8BA824FD,0xC0449E98,0xB743CC56,0x41FADDAC,0x141E9B1C,
0x8B937233,0x9B59DCA7,0xF1C871AD,0x6C678B4D,0x46617752,0xAAE49354,
0xCABE8156,0x6DOAC54C,0x680CA74C,0x5CD82B3F,0xA1C72A59,0x336EFB54,
0xD3B1A748,0xF4EB40D5,0x0ADB36CF,0x59FA1CE0,0x2C694FF9,0x5CE2F81A,
0x469B9E34,0xCE74A493,0x08B55111,0xEDED517C,0x1695D6FE,0xE37C7EC7,
0x57827B93,0x0E02A748,0x6E4A9C0F,0x4D840764,0x9DFFC45C,0x891D29D7,
0xF9AD0D52,0x3F663F69,0xD00A91B9,0x615E2398,0xEDBBC423,0x09397968,
0xE42D6B68,0x24C7EFB1,0x384D472C,0x3F0CE39F,0xD02E9787,0xC326F415,
0x9E135320,0x150CB9E2,0xED94AFC7,0x236EAB0F,0x596807A0,0x0BD61C36,
0xA29E8F57,0x0D8099A5,0x520200EA,0xD11FF96C,0x5FF47467,0x575C0B39,
0x0FC89690,0xB1FBACE8,0x7A957D16,0xB54D9F76,0x21DC77FB,0x6DE85CF5,
0xBFE7AEE9,0xC49571A9,0x7F1DE4DA,0x29E03484,0x786BA455,0xC26E2109,
0x4A0215F4,0x44BFF99C,0x711A2414,0xFDE9CDD0,0xDCE15B77,0x66D37887,
0xF006CB92,0x27429119,0xF37B9784,0x9BE182D9,0xF21B8C34,0x732CAD2D,
0xAF8A6A60,0x33A5D3AF,0x633E2688,0x5EAB5FD1,0x23E6017A,0xAC27A7CF,
0xF0FC5A0E,0xCC857A5D,0x20FB7B56,0x3241F4CD,0xE132B8F7,0x4BB37056,
0xDA1D5F94,0x76E08321,0xE1936A9C,0x876C99C3,0x2B8A5877,0xEB6E3836,
0x9ED8A201,0xB49B5122,0xB1199638,0xA0A4AF2B,0x15F50A42,0x775F3759,
0x41291099,0xB6131D94,0x9A563075,0x224D1EB1,0x12BB0FA2,0xFF9BFC8C,
0x58237F23,0x98EF2A15,0xD6BCCF8A,0xB340DC66,0x0D7743F0,0x13372812,
0x6279F82B,0x4E45E519,0x98B4BE06,0x71375BAE,0x2173ED47,0x14148267,
0xB7AB85B5,0xA875E314,0x1372F18D,0xFD105270,0xB83F161F,0x5C175260,
0x44FFD49F,0xD428C4F6,0x2C2002FC,0xF2797BAF,0xA3B20A4E,0xB9BF1A89,
0xE4ABA5E2,0xC912C58D,0x96516F9A,0x51561E77};
```



5、Dragon:S盒

■ 例: $S_1(0x01) = 0x393BCE6B$

■ 例: $S_2(0xFE) = 0x96516F9A$

■ 例:

$$G_1(0x010101FE) = 0x393BCE6B \oplus 0x393BCE6B \oplus 0x393BCE6B \oplus 0x96516F9A$$

■ 对每个 $S_i(x_j)$ ($i=1, 2, j=0, 1, 2, 3$), 将 x_j 所对应的十六进制数按 S_i 查表所得的32位数是输出。 S_1 和 S_2 合起来实质上形成 32×32 位的S盒。

2024



5、Dragon:密钥流

Dragon 所具有的 1024 位的 NFSR 被分为 32 个 32 位的字 B_i , $0 \leq i \leq 31$ 。每一轮中, 内部状态的 6 个字用来作为 F 函数的输入。这些字的索引是 0、9、16、19、30 和 31, 并且形成一个全部正的差分集合。另外, M 作为密钥流产生过程中的计数器。密钥流产生的每一轮都输出 64 位的字 k 。下面的算法给出了密钥流产生的一轮操作。

- 注意 32 个字及计数器也在变化

王卫华



5、Dragon:密钥流

■ 从 W_0 -- W_7 和 M 开始迭代，产生密钥流：

输入 = $\{ B_0 \parallel \dots \parallel B_{31} , M \}$

1、 $(M_L \parallel M_R) = M$

2、 $a = B_0, b = B_9, c = B_{16}, d = B_{19}, e = B_{30} \oplus M_L, f = B_{31} \oplus M_R$

3、 $(a', b', c', d', e', f') = F(a, b, c, d, e, f)$

4、 $B_0 = b', B_1 = c'$

5、 $B_i = B_{i-2}, 2 \leq i \leq 31$

6、 $M = M + 1$

7、 $k = a' \parallel e'$

输出 = $\{ k , B_0 \parallel \dots \parallel B_{31} , M \}$

2024



5、Dragon:安全性

- Dragon算法是基于安全和效率的思想设计的，它能够抵抗现今所有已知的密码攻击。Dragon算法采用的是非线性反馈和分组迭代相结合的方式，可以看作是分组密码OFB模式的一次进化，这种修改克服了分组密码OFB模式的缺点：输出密钥流同时反馈到内部状态（注意 a' 和 e' 并没有反馈回去）。
- n 基于生日悖论的碰撞攻击能够利用反馈的结果，但是，在Dragon算法中，这种类型的攻击被产生单独的输出和从更新函数的反馈字所阻。

2024



5、Dragon:安全性

- 具有小的内部状态的密码易受到时间/内存/数据的折衷攻击，克服这类攻击的一个最小的必备条件就是使得内部状态至少是设计安全的两倍。在Dragon算法中，一个大的内部状态使得这类攻击无法奏效。
- 为了增加猜测攻击和决定性攻击的难度，Dragon算法根据FPDS在非线性移位寄存器中选择了一些跳跃点。
- 总之， Dragon算法目前认为是安全的。

王卫华



5、Dragon:思考

- dragon的密钥长度是 [填空1]，初始向量的长度为 [填空2]，算法的输出是基于 [填空3] 而设计。



3.3.5 序列密码的总结

- 密钥流的产生看似很复杂，但用硬件实现很简单：
 - 产生速度与计算机时钟速度相当（可与语音同步）
 - 从一个64比特密钥可产生无穷多密钥流
 - 最终会产生密钥流循环
- 因为过去基于软件的密码系统不能产生高速比特流，序列密码曾十分辉煌，当今，基于软件的密码系统的出现，使**分组密码成为主流**。

王立华



1、序列密码总结

- **基于移位寄存器构造序列密码是传统**的方法，但研究仍在继续，eStream最终选择的算法中就有不少基于移位寄存器构造的算法。
- 由于线性移位寄存器的理论已经成熟，特别是 m 序列具有良好的伪随机性，所以驱动部分的设计相对容易，于是**非线性组合部分的设计**便成为密钥流生成器的重点与难点。
- 而在二元情况下，非线性组合部分就可用一个布尔函数表示。于是密钥流生成器的研究归结为**布尔函数**的研究。

王华



1、序列密码总结

- 序列密码本身是**对称算法**的一个分支，与分组算法在加密特征上有明显区别。
- 通常**分组算法可以转换成序列密码**，但**反过来则不行**。
- 基于移位寄存器设计的序列密码有着速度上优势。但**非线性反馈移位寄存器**的输出通常不易计算其周期，有时通过增加线性混合来解决。

2022/4/9



思考:

• 课后习题:

➤1

➤3

➤6

➤8

➤9

例、已知 $f(x) = x^6 \oplus x \oplus 1$ 是6次本原多项式, $\underline{a}$ 是 $f(x)$ 生成的m序列,

(1) $\underline{a}$ 的周期是多少?

(2) $\underline{a}$ 在的一个周期内, 0、1各出现多少次?

(3) $\underline{a}$ 在的一个周期内, 游程分布如何?

王华